The VLDB Journal (2026) 35:25
https://doi.org/10.1007/s00778-026-00977-5

REGULAR PAPER

Efficiently querying connected components in large temporal graphs via scalable and maintainable indices

Yuyang Xia[1] · Haoxuan Xie[2] · Yixiang Fang[1] · Wensheng Luo[3] · Chenhao Ma[1] · Dong Wen[4]

Received: 7 April 2025 / Revised: 7 December 2025 / Accepted: 1 April 2026 / Published online: 8 May 2026
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

In many real-world applications, the relationships between entities can be modeled as temporal graphs, where each edge is associated with a timestamp representing the interaction time. As a fundamental problem in network science, the connected component (CC) query has received tremendous research attention. Existing works on CC queries in the temporal graph find sets of vertices that are either connected in every timestamp of a time interval, or connected by paths with edges of increasing timestamps (i.e., time-respecting paths). However, these temporal constraints are too strict for applications without needing time-respecting paths. In this paper, we relax the above constraints by introducing a novel CC model, called window-CC, for both the undirected and directed temporal graphs in a given time window. We first propose online algorithms to query the window-CC, and further develop efficient and scalable index-based solutions. Besides, to handle queries on large dynamic temporal graphs, we also design efficient algorithms to dynamically update indices for new edges. Experimental results on real-world large-scale temporal graphs (both undirected and directed) demonstrate that our solutions achieve triple advantages:

1. Our best index-based query algorithms are up to four and two orders of magnitude faster than the two online algorithms, respectively.
2. Compared with the baseline indices, our optimized indices cost much less space in both theory and practice.
3. Experiments also demonstrate the high efficiency of our index construction and update algorithms.

Keywords Temporal graph/network · Connected component · Window-CC

1 Introduction

In many real-world applications, the relationships between entities can be modeled as temporal graphs, where each edge is associated with a timestamp representing the interaction time. Research on temporal graphs has recently attracted much attention from both industry and research communities [37, 52, 67]. Figure 1 presents examples of undirected and directed temporal graphs, where the numbers on the edges denote the occurring time of the edges. For example, in Figure 1(a) , the edge between vertices v_1 and v_2 denotes that they have an interaction (e.g., v_1 and v_2 confirms a transaction) at timestamp 0; in Figure 1(b) , the edge between vertices v_2 and v_3 denotes that they have an interaction (e.g., v_2 sends a message to v_3) at timestamp 1.

In this paper, we study the problem of connected component (CC) queries on temporal graphs, and the general goal

These authors contributed equally to this work

✉ Yixiang Fang
fangyixiang@cuhk.edu.cn

Yuyang Xia
yuyangxia@link.cuhk.edu.cn

Haoxuan Xie
haoxuan001@e.ntu.edu.sg

Wensheng Luo
luowensheng@hnu.edu.cn

Chenhao Ma
machenhao@cuhk.edu.cn

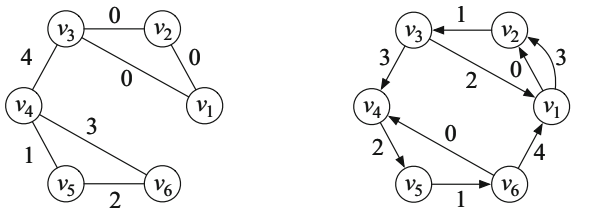
Dong Wen
dong.wen@unsw.edu.au

[1] The Chinese University of Hong Kong, Shenzhen, Shenzhen, China

[2] Nanyang Technological University, Singapore, Singapore

[3] Hunan University, Changsha, China

[4] University of New South Wales, Sydney, Australia



(a) An undirected graph (b) A directed graph

Fig. 1 Two example temporal graphs

is to find all the CCs in a query time window from a temporal graph. As a fundamental structure in the graph, the CC is a maximal set of vertices such that each pair of vertices is mutually reachable in the graph. It has found various real applications, such as community detection [20, 44], PPI network analysis [13, 24], and network routing protocols [3, 35]. Most of the existing works about CC focus on conventional static graphs, including distributed BFS search [5] and distributed Union-Find algorithms [15, 50], but they cannot be applied to temporal graphs due to the temporal edges. Recently, some works have attempted to study the CC query on temporal graphs. For example, Vernet et al. [66] proposed the persistent CC model for temporal graphs, which is a set of vertices that are connected in every timestamp of a time window. Bhadra and Ferreira [7, 8] introduced the temporal CC model, which is a set of vertices such that each pair of vertices is connected by a path with edges of increasing timestamps (i.e., time-respecting path [12, 53]). However, these temporal constraints are too strict for applications without needing such time-respecting paths.

To tackle the issues of existing works, we relax the temporal constraints by introducing a novel CC model, called window-CC, for both the undirected and directed temporal graphs in a time window $[t_s, t_e]$. Specifically, given a temporal graph G and a time window $[t_s, t_e]$, we first introduce the concept of the projected graph, which is a static graph formed by edges in G with timestamps in $[t_s, t_e]$. We then formulate the window-CC of the undirected temporal graph as a set of vertices that are mutually reachable by undirected paths in the projected graph. The window-CC can also be extended as window-SCC for the directed temporal graph, which is a set of vertices that are mutually reachable by directed paths in the projected graph. Based on the concepts above, we propose the window-CC and window-SCC queries, which aim to find all the window-CCs and window-SCCs from the undirected and directed temporal graphs, respectively.

Applications. The window-CC and window-SCC queries can be used in many real applications. Here we just name a few:

- **Infectious disease tracking.** To track the transmission of infectious diseases (e.g., transmission of Covid-19[1]), researchers often model the interactions between people as an undirected temporal graph, where each vertex denotes a person and each temporal edge between two vertices means they have an interaction (e.g., handshake) at a specific timestamp. The window-CCs of the graphs can be used to reveal the possible infected people. For example, if a patient is tested positive for Covid-19 at timestamp t, then all the people within the patient's window-CC of time window $[t-d, t]$ have been potentially infected, where d is the incubation period of Covid-19 (usually 2 weeks), since they have direct or indirect contacts with the patient. Note that here we mainly consider the projected graph of the incubation period, but do not need the strict time-respecting paths since the timestamps of edges may not be accurately collected in practice. For instance, contact timestamps may be incomplete or imprecise at the very early stage of a disease breakout. Window-CCs therefore offer a more robust characterization of possible exposure.
- **Active community analysis.** In social networks, users are often actively involved in different communities during different periods, where a community is often an SCC [20, 44]. For instance, the retweet relationships between Twitter users can be modeled as a directed temporal graph, where each vertex denotes a user and each temporal edge between two vertices means one retweets a message of the other at a specific timestamp. By finding the window-SCCs in different time windows (e.g., different weeks) from the graph, we can track the active communities that a user participates in. Thus, various recommendation tasks can be performed in different time periods by using the community-based A/B testing [18], which assigns different treatments (e.g., friend invitation or product recommendation) to the user, as widely investigated by LinkedIn [29] and Alipay [9].
- **Network anomaly detection.** In E-commerce, the money transfer transactions between users can be modeled as a temporal graph. Since cycles may reveal anomaly behaviors (e.g., credit card fraud) [55], the window-CCs/SCCs can be applied to detecting them. For instance, by computing window-CCs or window-SCCs over recent time windows, one can detect groups of users that suddenly become densely or cyclically interconnected within a short period. Besides, we will show later that we have used window-SCCs to identify anomaly DBLP data in Section 6.4.

[1] https://www.cdc.gov/museum/pdf/cdcm-pha-stem-lesson-contact-tracing-lesson.pdf

Example 1.1 We utilize Figure 1(a) to illustrate how window-CC captures real-world applications. In the context of **infectious disease tracking**, assume vertex v_4 is a patient who tests positive at time $t = 4$ with an incubation window of $[t - 2, t] = [2, 4]$. Within this window, the set $\{v_3, v_4, v_5, v_6\}$ forms a window-CC connected via edges $(v_3, v_4, 4)$, $(v_4, v_6, 3)$, and $(v_5, v_6, 2)$. This indicates that v_3, v_5, and v_6 may have been infected by patient v_4. Notably, the path $v_4 \to v_6 \to v_5$ is not a time-respecting path because the timestamps decrease $(3 \to 2)$. However, window-CC includes v_5 to provide a more robust analysis of potential exposure.

Online solutions. Online solutions can be designed using existing CC computation algorithms. Specifically, given an undirected temporal graph and a time window $[t_s, t_e]$, we first collect all the edges with timestamps in this window, and then compute all the CCs from the graph projected by these edges using a CC computation algorithm (e.g., BFS or DFS). Similarly, for the directed temporal graph, we can compute all the SCCs using an SCC computation algorithm (e.g., [26, 60, 64]). However, these solutions require accessing all the edges in the window, so the efficiency and scalability are limited, especially when the window is wide.

In this paper, we aim to design index-based solutions for efficient window-CC and window-SCC queries on large temporal graphs. A naive index is to pre-compute and store the answer for any time window. Although this approach has high query efficiency, the index itself takes $O(nt_{max}^2)$ space cost, where n denotes the number of vertices and t_{max} is the number of distinct time slots in the graph, making the space overhead render this approach impractical.

Index-based solutions. For the undirected temporal graph, we can build an index by compressing all the CCs in the time windows with the same start time. Specifically, consider a fixed start time t_s. Initially, we regard each vertex v as a single CC and assign it a label $L(v)$ indicating the CC that contains it. Then, we sequentially consider the edges in $[t_s, t_{max}]$ in chronological order and merge the disjoint CCs connected by each edge. Whenever there is a merge process between two disjoint CCs, we update the label of vertices in the smaller CC with the label of the larger CC and record the update time. After the above process, each vertex is associated with a list of $\langle label, time \rangle$ pairs, which can serve as an index, termed as U-baseline. The index costs $O(n \log n \cdot t_{max})$ space and can be built in $O((m + n \log n)t_{max})$ time, where m is the number of edges.

Although the U-baseline index is better than the naive index, it is still costly in time and space when t_{max} is large. We further propose the TSF-index, or temporal spanning forest (TSF) index, by exploiting the idea that the CCs of an undirected graph can be represented by its spanning trees. Specifically, we first introduce the concept of TSF, which preserves the spanning trees of a projected undirected graph by an edge set. We then observe that there are two kinds of overlapping relationships among the TSFs over different time windows. The first one reveals the nested relationships of TSFs sharing the same start time, while the second one shows the overlapping relationships of TSFs for multiple start times. Based on the observation, we design the TSF-index, whose time and space cost are much less than those of U-baseline index. Moreover, given the dynamic nature of temporal graphs, it is crucial to maintain the TSF-index efficiently to accommodate new edge insertions. Therefore, we design an efficient method to update our TSF-index without rebuilding the index from scratch for every update.

For the directed temporal graph, inspired by TSF-index, we develop an index called D-baseline by building a forest structure for each start time t_s, where two vertices are in the same tree iff they are in the same SCC. However, unlike the undirected graph, the SCCs of a directed graph cannot be derived by a directed spanning tree [27], so we cannot use the directed spanning tree to represent an SCC. Instead, we propose to create some undirected edges to denote the SCCs. For each start time t_s, we run SCC algorithms on the projected graph $G_{[t_s,t]}$ for each $t \in [t_s, t_{max}]$ in chronological order, and create the aforementioned forest structure. Since the forest contains at most $(n - 1)$ edges, the index costs $O(nt_{max})$ space and can be built in $O((m + n)t_{max}^2)$ time.

The D-baseline index is still costly in both time and space cost when t_{max} is large. To alleviate these issues, we further develop another novel index, called RES-index or reconstruction edge set (RES) index. The key idea is that for any SCC with k vertices in the directed graph, we theoretically prove that it can be represented by a set of $2(k - 1)$ edges in the original SCC, which is termed as the reconstruction edge set (RES). Inspired by the two kinds of overlapping relationships among TSFs above, we further identify two kinds of overlapping relationships among the RES's over different time windows. We further develop the RES-index which takes the same space cost as TSF-index. In addition, we develop a RES-index maintenance algorithm for large dynamic directed temporal graphs. Tables 1 and 2 show the complexities of the solutions for undirected and directed temporal graphs respectively.

We would like to remark that there is a trade-off between the online and index-based solutions. While the index-based solutions need some extra time cost for offline index construction, they outperform online solutions when the number of queries is large, because their offline processing time cost could be amortized. The experimental results on real-world temporal graphs show that our solutions are highly efficient. In particular, the queries based on the TSF-index and RES-index are up to four and two orders of magnitude faster than the two online algorithms. Besides, these two indices cost much less space than the baseline indices.

Table 1 Solutions for undirected temporal graphs

	U-online	U-baseline	TSF-index
Query time	$O(m+n)$	$O(n \log\log n)$	$O(n+\sqrt{t_{max}})$
Index space	∅	$O(n \log n \cdot t_{max})$	$O(m)$
Index time	∅	$O((m+n\log n)t_{max})$	$O(m \log n)$
Maintenance	∅	$O(n \cdot t_{max})$	$O(\log n)$

Table 2 Solutions for directed temporal graphs

	D-online	D-baseline	RES-index
Query time	$O(m+n)$	$O(n)$	$O(n+\sqrt{t_{max}})$
Index space	∅	$O(n \cdot t_{max})$	$O(m)$
Index time	∅	$O((m+n)t_{max}^2)$	$O(mt_{max})$
Maintenance	∅	$O((m+n)t_{max})$	$O(mt_{max})$

In summary, our key contributions are as follows.

- We introduce the problems of querying window-CCs and window-SCCs on undirected and directed temporal graphs, respectively, which have not been studied in the literature yet.
- We propose several index-based solutions to support the efficient queries of window-CCs and window-SCCs. The space costs of the best indices are linear to the sizes of the graphs.
- For dynamic temporal graphs where temporal edges are inserted or deleted, we develop efficient index maintenance algorithms for the TSF-index and partial support for the RES-index.
- We generalize the TSF-index and the RES-index to support both timestamped and interval-based temporal graphs.
- We perform various experiments on real-world and synthetic datasets, demonstrating the high effectiveness, efficiency, and scalability of our solutions.

Remark. An earlier version of this work has been published in the SIGMOD 2023 conference [71]. Our newly added contributions to this paper are as follows: 1) For undirected temporal graphs, we have optimized the structure of our previous TSF-index, which allows more efficient queries, and developed both novel fast TSF-index construction and maintenance algorithms. 2) For directed temporal graphs, we have optimized the structure of our previous RES-index, which allows more efficient queries, and developed a novel fast RES-index construction algorithm. 3) For temporal graphs where edges are associated with a time interval instead of a timestamp, we extend the TSF-index and the RES-index to support them without introducing additional complexity overheads. 4) We have also conducted extensive experiments on larger datasets to verify the efficiency.

Outline. We formulate our research problems and present online algorithms in Section 2. In Sections 3 and 4, we present the index-based algorithms for undirected and directed temporal graphs, respectively. We report experimental results in Section 6. We review the related work in Section 7 and conclude in Section 8.

Supplementary information. Due to space limitations, all detailed proofs of the lemmas in this paper are omitted from the main text and are provided in the supplementary information. https://github.com/ForwardStar/spannedCC/blob/main/Appendix.pdf

2 Preliminaries

We first formally present the window-CC and window-SCC queries in Section 2.1, and then show their online solutions in Section 2.2.

2.1 Problem definitions

Consider an undirected temporal graph $G = (V, E)$, where V and E denote the sets of vertices and edges, respectively. Let $|V| = n$ and $|E| = m$. Each edge $e \in E$ is a triplet (u, v, t) with $u \in V$, $v \in V$ and $t \in \mathbb{N}$ representing the interaction timestamp between u and v. W.l.o.g., we assume that the timestamps of edges are consecutive integer values in the range $[0, t_{max}]$, implying that $m \geq t_{max}$.

Definition 2.1 (*Projected graph*) The projected graph of an undirected temporal graph $G = (V, E)$ over a time window $[t_s, t_e]$ is $G_{[t_s,t_e]}=(V, E_{[t_s,t_e]})$, where $E_{[t_s,t_e]} = \{(u, v)|(u, v, t) \in E \wedge t \in [t_s, t_e]\}$.

Before formulating the concept of window-CC, we present the definition of span-reachability [67].

Definition 2.2 (*Span-reachability* [67]) Given an undirected temporal graph $G = (V, E)$, two vertices u, v and a time window $[t_s, t_e]$, u span reaches v in $[t_s, t_e]$, denoted by $u \rightsquigarrow_{[t_s,t_e]} v$, if u is connected to v via a path in the projected graph $G_{[t_s,t_e]}$.

Definition 2.3 (*Window-CC*) Given an undirected temporal graph $G = (V, E)$ and a time window $[t_s, t_e]$, the window-CC is a maximal set of vertices S, such that for any two vertices $u, v \in S$, $u \rightsquigarrow_{[t_s,t_e]} v$.

Example 2.1 In the undirected temporal graph of Figure 1(a) , we have $v_1 \rightsquigarrow_{[0,4]} v_6$ but $v_1 \not\rightsquigarrow_{[0,3]} v_6$.

For the time window [0, 4], there is exactly one window-CC $\{v_1, v_2, v_3, v_4, v_5, v_6\}$, while for the time window [0, 3], there are two window-CCs, i.e., $\{v_1, v_2, v_3\}$ and $\{v_4, v_5, v_6\}$.

Problem 2.1 (*Window-CC query*) Given an undirected temporal graph $G = (V, E)$ and a time window $[t_s, t_e]$, find all the window-CCs in this time window.

For the directed temporal graph, the definitions of projected graphs and span-reachability can be similarly defined as shown in [67], so we omit the details. The concept of the window-CC can be reformulated as window-SCC, defined as follows.

Definition 2.4 (*Window-SCC*) Given a directed temporal graph $G = (V, E)$ and a time window $[t_s, t_e]$, the window-SCC is a maximal set of vertices S, such that for any two vertices $u, v \in S$, $u \rightsquigarrow_{[t_s,t_e]} v$ and $v \rightsquigarrow_{[t_s,t_e]} u$.

Problem 2.2 (*Window-SCC query*) Given a directed temporal graph $G = (V, E)$ and a time window $[t_s, t_e]$, find all the window-SCCs in this time window.

Example 2.2 Consider the directed temporal graph in Figure 1(b) .

For the time window [0, 4], the whole graph is a window-SCC.

For the time window [0, 2], there are two window-SCCs, which are $\{v_1, v_2, v_3\}$ and $\{v_4, v_5, v_6\}$.

2.2 Online algorithms

Before presenting the online algorithms, we introduce a technique for shrinking the size of the undirected temporal graph without affecting the final query result. Since the reachability information of a static undirected graph can be maintained by a spanning forest with at most $(n-1)$ edges, we can construct a spanning forest for each $G_{[t,t]}$ with each $t \in [0, t_{max}]$, and then replace E with the edges in the spanning forests. In practice, since the shrinking technique above can reduce the number of edges significantly, we use it to preprocess all the undirected temporal graphs by default. Note that for the directed graph, since the directed spanning forest [27] cannot capture the reachability information, the shrinking technique above cannot be used directly.

As aforementioned, there are some existing algorithms for querying CCs or SCCs in conventional static graphs. For querying CCs in an undirected graph, the Breadth-First Search (BFS), Depth-First Search (DFS), and Union-Find data structure can give solutions in $O(m+n)$ time cost. For querying SCCs in a directed graph, various algorithms have been discovered to process in $O(m+n)$ time [26, 60, 64]. These algorithms can be used for answering our window-CC queries on temporal graphs. The general idea is that we can first build a projected graph $G_{[t_s,t_e]}$, and then run an existing CC computation algorithm above if $G_{[t_s,t_e]}$ is undirected, or an existing SCC computation algorithm above if $G_{[t_s,t_e]}$ is directed. For lack of space, we omit the details here. We denote the online algorithms for processing undirected and directed temporal graphs by U-online and D-online respectively.

Lemma 2.1 *The running time of the online window-CC and window-SCC query algorithms are bounded by* $O(m_{[t_s,t_e]} + n)$.

Unlike the conventional static graph, the temporal graph often has a relatively large number of edges due to the unlimited number of timestamps. Therefore, although the online query algorithms take linear time cost as stated by Lemma 2.1, they are still inefficient and unscalable for processing large temporal graphs. In this paper, we focus on developing efficient index-based solutions.

We would also like to note that the window-CC is defined based on span-reachability, so our solutions can also be used to answer the span-reachability queries in undirected temporal graphs [67].

3 Index-based solutions for undirected temporal graphs

As mentioned in Section 1, the naive index takes $O(nt_{max}^2)$ space cost, which is impractical if t_{max} is large, so we do not further consider it in the paper. In the following, we first present two index-based solutions, and the best one only costs $O(m)$ space.

3.1 A baseline index-based solution

3.1.1 Index overview

In this section, we propose a nontrivial index named U-baseline index, and the main idea is that for each start time, we try to record a set of "labels" for each vertex, such that the window-CCs in different time windows are well-preserved.

Specifically, consider a sub-problem where the start time t_s is fixed and the goal is to answer a query with time window $[t_s, t_e]$ where $t_e \in [t_s, t_{max}]$. We observe that for the time windows $[t_s, t_s], [t_s, t_s+1], \cdots, [t_s, t_{max}]$, the corresponding window-CCs would only be merged but never split, when t_e increases from t_s to t_{max}. Thus, by considering all the edges

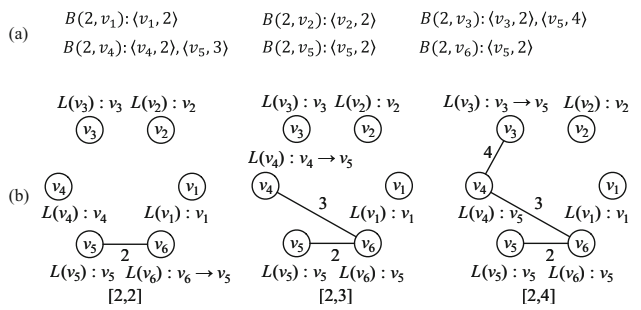


Fig. 2 The U-baseline index by anchoring $t_s = 2$

in $[t_s, t_{max}]$ in chronological order, we can label the window-CCs and keep track of label changes when they are merged. We now formally define the vertex label.

Definition 3.1 (*Vertex label*) Given an undirected graph, the label of each vertex v, $L(v)$, denotes the CC containing it. Initially, each vertex v forms a single CC with a label $L(v) = v$.

We denote the vertex set of the CC containing v by $S(v)$, i.e., $S(v) = \{u \in V | L(u) = L(v)\}$. By considering all the edges in chronological order, we can merge the window-CCs incrementally. Initially, each vertex v forms a single CC with $L(v) = v$. Then, for each edge (u, v, t), if u and v are not in the same window-CC, we merge the two window-CCs of u and v, by adding vertices of the small one into the large one, with the following union operation:

Definition 3.2 (*Union*) For an edge (u, v, t), if $|S(u)| > |S(v)|$, then we set $L(w) = L(u)$ for all $w \in S(v)$, and update $S(u) = S(u) \cup S(v)$; otherwise, we set $L(w) = L(v)$ for all $w \in S(u)$, and update $S(v) = S(u) \cup S(v)$.

In the above process, whenever we merge two window-CCs, we can record some $\langle label, time\rangle$ pairs for vertices whose labels are changed. As a result, we obtain an index structure, which contains such pairs for each start time and each vertex, denoted by:

$$B(t_s, v) = (\langle l_1, t_1\rangle, \langle l_2, t_2\rangle, \cdots)$$

Example 3.1 Consider the graph in Figure 1(a) . By anchoring the start time $t_s = 2$, we can build the index structure in Figure 2(a), where the process of merging CCs is depicted in Figure 2(b).

Lemma 3.1 *For each anchored start time t_s, the label of each vertex is changed at most* $(\log n)$ *times.*

Lemma 3.2 *The baseline index costs $O(n \log n \cdot t_{max})$ space.*

3.1.2 Query processing

Based on the index above, to answer a query with the time window $[t_s, t_e]$, we first retrieve $B(t_s, v)$ for each vertex $v \in V$. Then, for each vertex v, we find the pair $\langle label, time\rangle$ corresponding to the query time window. Finally, we build an undirected graph and compute the CCs from it. Algorithm 1 presents the detailed steps. It initializes a graph G' with only n vertices (line 1). Then, for each vertex v, it binary searches the $\langle l_k, t_k\rangle$ satisfying $t_k \leq t_e$ and $t_{k+1} > t_e$ from $B(t_s, v)$ (lines 2-3), and adds an edge (v, l_k) to G' (line 4). Note that after the process above, G' has $n - 1$ edges. Finally, it finds and returns all the CCs in G' (line 5).

Algorithm 1: U-baseline-query(B, $[t_s, t_e]$)

Input: the index B and query time window $[t_s, t_e]$;
Output: all the window-CCs in $G_{[t_s,t_e]}$;
1 $G' \leftarrow (V, E')$ where $E' = \emptyset$;
2 **for** $v \in V$ **do**
3 $\quad \langle l_k, t_k\rangle \leftarrow$ binary search a pair $\langle l_k, t_k\rangle$ such that $t_k \leq t_e$ and $t_{k+1} > t_e$ from $B(t_s, v)$;
4 $\quad$ add an edge (v, l_k) between v and l_k into G';
5 **return** all the CCs computed from G';

Lemma 3.3 *Algorithm 1 answers a query in $O(n \log \log n)$ time.*

3.1.3 Index construction

To build the index, for each start time t_s, we sequentially consider the edges in chronological order, during which we merge the CCs and record the $\langle label, time\rangle$ pairs. Algorithm 2 shows the details. For each start time t_s, we first initialize $L(v)$, $S(v)$, and $B(t_s, v)$ for each vertex v (lines 1-4). We then sequentially consider the edges in chronological order, merge window-CCs, and record the $\langle label, time\rangle$ pairs (lines 5-14). For each edge (u, v, t), if the two end vertices are not in the same CC, we merge the vertices of the small CC into the large one and change their labels, during which the $\langle label, time\rangle$ pairs are recorded.

Lemma 3.4 *Algorithm 2 takes $O((m + n \log n)t_{max})$ time cost.*

In addition, the U-baseline index can be easily updated without re-building it from scratch, when a new edge $(u, v, t_{max} + 1)$ is added into the graph. Specifically, we can design a revised version of Algorithm 2, by ranging t_s from 0 to $(t_{max} + 1)$ in its outer loop. When $t_s \in [0, t_{max}]$, instead of the trivial initialization in lines 2-4, we re-use the sets L, S with the previously derived $B(t_s, \cdot)$, and only consider the newly added edge by running lines 5-14; when $t_s = t_{max} + 1$, we directly follow the original procedure. The maintenance

Algorithm 2: U-baseline-construct(G)

```
Input: an undirected temporal graph G = (V, E);
Output: the index B(t_s, v) for all 0 ≤ t_s ≤ t_max and all v ∈ V;
1  for t_s ∈ [0, t_max] do
2      for v ∈ V do
3          L(v) ← v, S(v) ← {v};
4          B(t_s, v) ← (⟨v, t_s⟩);
5      for (u, v, t) ∈ E ∧ t ≥ t_s do
6          if L(u) ≠ L(v) then
7              if |S(u)| ≥ |S(v)| then
8                  S(u) ← S(u) ∪ S(v);
9                  for w ∈ S(v) do
10                     L(w) ← L(u), S(w) ← S(u);
11                     delete all ⟨l, t⟩ pairs in B(t_s, w);
12                     append ⟨L(w), t⟩ to B(t_s, w);
13             else
14                 merge S(u) into S(v) and update labels (similar to
                   lines 8-12);
```

cost per new edge is $O(nt_{max})$, since the algorithm iterates over $O(t_{max})$ start times, and for each t_s, processing the additional edge in lines 5–14 requires at most $O(n)$ time.

3.2 An advanced index-based solution

Generally, the solution using the U-baseline index performs well on small-to-moderate-sized graphs, but the index space cost may still be too large, especially when t_{max} is large. To further reduce the space and time cost of indexing, we propose an advanced index, called temporal spanning forest (TSF) index, or TSF-index, by exploiting the idea that on an undirected graph, each CC corresponds to a spanning tree of its spanning forest.

3.2.1 The concept of TSF and its properties

We first introduce the novel concept of TSF.

Definition 3.3 (*TSF*) Given an undirected temporal graph $G = (V, E)$ and a time window $[t_s, t_e]$, the TSF over $[t_s, t_e]$ is the minimum edge set $\Psi_{[t_s,t_e]} \subseteq E$, such that all the CCs of the graph $G' = (V, \Psi_{[t_s,t_e]})$ are exactly the window-CCs of $G_{[t_s,t_e]}$.

By Definition 3.3, the TSF has the minimum number of edges, implying that the edges form the spanning trees. Next, we show that there exist two kinds of overlapping relationships among different TSFs, allowing us to design a space-efficient index.

- **Nested relationship by anchoring a start time.** We formalize this relationship by Lemma 3.5.

Lemma 3.5 *Given an undirected temporal graph G and an anchored start time t_s, there exists a chain of TSFs such that:*

$$\Psi_{[t_s,t_s]} \subseteq \Psi_{[t_s,t_s+1]} \subseteq \cdots \subseteq \Psi_{[t_s,t_{max}]}$$

By Lemma 3.5, for each anchored t_s, we can compress all the TSFs $\Psi_{[t_s,t_s]}, \Psi_{[t_s,t_s+1]}, \cdots, \Psi_{[t_s,t_{max}]}$ using $O(n)$ space cost by only keeping the newly added edges for each TSF. However, if we directly compress all the TSFs for each $t_s \in [0, t_{max}]$ as an index, it will take $O(nt_{max})$ space cost, which is still costly when t_{max} is large. To overcome the enormous space cost caused by t_{max}, we observe and exploit another overlapping relationship among TSFs.

- **Overlapping relationship for multiple start times.** We begin with an important observation in Lemma 3.6, which allows us to compress all the TSFs by using only $O(m)$ space.

Lemma 3.6 *Given an undirected temporal graph G, there exist $(t_{max}+1)$ TSFs, i.e., $\Psi_{[0,t_{max}]}, \Psi_{[1,t_{max}]}, \cdots, \Psi_{[t_{max},t_{max}]}$, such that for each edge (u, v, t), there is a chain starting from some $\widehat{t} \in [0, t]$:*

$$\Psi_{[\widehat{t},t_{max}]}, \Psi_{[\widehat{t}+1,t_{max}]}, \cdots, \Psi_{[t,t_{max}]}$$

satisfying (1) each TSF in the chain contains the edge and (2) the edge is excluded in any TSFs that are not in the chain.

By Lemma 3.6, for each edge (u, v, t), we can identify an *appearing time interval* $[\widehat{t}, t]$ such that for any $t' \in [\widehat{t}, t]$, the TSF over $[t', t_{max}]$ contains it.

3.2.2 Index overview

After processing each edge and obtaining its appearing time interval, we can reverse the organization of the edges and their appearing time intervals, such that for each time interval $[t_i, t_j]$ where $0 \le t_i \le t_j \le t_{max}$, there is a set of edges corresponding to it, denoted by $F(t_i, t_j)$.

To answer a query with time window $[t_s, t_e]$, our goal is to identify all window-CCs in $G_{[t_s,t_e]}$. Specifically, given a query time interval $[t_s, t_e]$, we need to fetch all $F(t_i, t_j)$ with $t_i \in [0, t_s]$ and $t_j \in [t_s, t_e]$. A naive approach is to iterate through every timestamp $t_j \in [t_s, t_e]$ and, for each t_j, find all non-empty $F(t_i, t_j)$ where $t_i \in [0, t_s]$. However, this linear scan results in a time complexity of $O(n + t_{max})$, which becomes the performance bottleneck when t_{max} is large.

To reduce the query time complexity, the TSF-index organizes all non-empty $F(t_i, t_j)$ using two complementary data structures to serve various query ranges:

– **Point array** ($F_p[]$): For each timestamp $x \in [0, t_{max}]$, $F_p[x]$ stores all edge sets $F(t_i, t_j)$ with $t_j = x$.

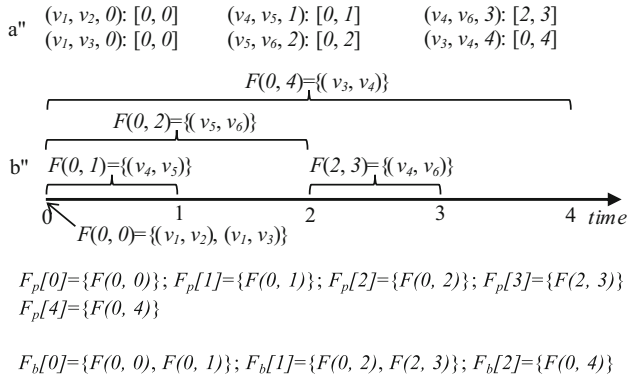


Fig. 3 The TSF-index for the graph in Figure 1(a)

– **Block array** ($F_b[]$): For each integer $y \in [0, \lfloor\sqrt{t_{max}}\rfloor]$, $F_b[y]$ stores $F(t_i, t_j)$ with $t_j \in [y\sqrt{t_{max}}, (y+1)\sqrt{t_{max}})$.

Within both arrays, edge sets are maintained in ascending order of t_i to facilitate efficient retrieval.

The idea of setting each block array responsible for $O(\sqrt{t_{max}})$ timestamps ensures the query time complexity to be $O(n + \sqrt{t_{max}})$. Lemma 3.8 further depicts it.

Example 3.2 Figure 3(a) presents the appearing time interval for each edge in the graph in Figure 1(a) . Based on the appearing time intervals, Figure 3(b) presents the edge sets F and the arrays $F_p[]$ and $F_b[]$ in TSF-index. Specifically, since $\sqrt{t_{max}} = 2$, $F_b[0]$ stores $F(t_i, t_j)$ with $t_j \in [0, 2)$, $F_b[1]$ stores $F(t_i, t_j)$ with $t_j \in [2, 4)$, and $F_b[2]$ stores $F(t_i, t_j)$ with $t_j \in [4, 6)$.

Lemma 3.7 *The TSF-index costs $O(m)$ space.*

3.2.3 Query processing

Algorithm 3 presents the details of how to fetch the required $F(t_i, t_j)$ through the TSF-index. It initializes a new static graph G' with the original n vertices (line 1). Then, it adds the edges of all $F(t_i, t_j)$ with $t_i \le t_s$ and $t_s \le t_j \le t_e$ into G' to form the TSF over $[t_s, t_e]$, which is done by enumerating $F_p[]$ and $F_b[]$ (lines 2-12). Specifically, considering $l = \lfloor\frac{t_s}{\sqrt{t_{max}}}\rfloor + 1$ and $r = \lfloor\frac{t_e}{\sqrt{t_{max}}}\rfloor$, we divide the target edge sets $F(t_i, t_j)$ with $t_i \le t_s$ and $t_s \le t_j \le t_e$ into three parts:

1. $F(t_i, t_j)$ with $t_i \le t_s$ and $t_j \in [t_s, l\sqrt{t_{max}})$.
2. $F(t_i, t_j)$ with $t_i \le t_s$ and $t_j \in [l\sqrt{t_{max}}, r\sqrt{t_{max}})$.
3. $F(t_i, t_j)$ with $t_i \le t_s$ and $t_j \in [r\sqrt{t_{max}}, t_e]$.

We then fetch $F(t_i, t_j)$ in the first and third parts from $F_p[]$ (lines 4-6, 10-12) and $F(t_i, t_j)$ in the second part from $F_b[]$ (lines 7-9). Finally, it finds and returns CCs on G' as the window-CCs (line 13).

Algorithm 3: TSF-query($F_p[\,], F_b[\,], t_s, t_e$)

Input: the TSF-index $F_p[\,]$ and $F_b[\,]$, and query time window $[t_s, t_e]$;
Output: all the window-CCs in $G_{[t_s,t_e]}$;

```
1  G' ← (V, E') where E' = ∅;
2  l ← ⌊ts/√tmax⌋ + 1;
3  r ← ⌊te/√tmax⌋;
4  for tj ∈ [ts, l√tmax) do
5      for F(ti, tj) ∈ Fp[tj] ∧ ti ∈ [0, ts] do
6          E' ← E' ∪ F(ti, tj);
7  for x ∈ [l, r) do
8      for F(ti, tj) ∈ Fb[x] ∧ ti ∈ [0, ts] do
9          E' ← E' ∪ F(ti, tj);
10 for tj ∈ [r√tmax, te] do
11     for F(ti, tj) ∈ Fp[tj] ∧ ti ∈ [0, ts] do
12         E' ← E' ∪ F(ti, tj);
13 return all the CCs computed from G' = (V, E');
```

Lemma 3.8 *Algorithm 3 answers a query in $O(n + \sqrt{t_{max}})$ time.*

Note that in our previous work [71], we claimed that the proposed query algorithm runs in $O(n)$ time by assuming $t_{max} \le n$. However, upon further analysis, we have found that when $t_{max} > n$, the time complexity degrades to $O(n+t_{max})$. In this paper, our novel query algorithm improves the time complexity under this case to $O(n + \sqrt{t_{max}})$.

Additionally, finding edges from the TSF-index can be viewed as a 3-sided Concurrent Range Reporting problem with L=1 [1]. Therefore, we can also reduce the theoretical time complexity of the edge-finding process to $O(n + \log t_{max})$ by replacing the TSF-index with the priority search trees [51] to manage the edge sets F. Although the time complexity seems lower, we show in Section 6.2.1 that the practical efficiency of our Algorithm 3 is slightly better than that using the priority search tree. The reason is that in real-world datasets, the difference between $\sqrt{t_{max}}$ and $\log t_{max}$ is small, and the constant factor of the TSF-index is much smaller compared to that of the priority search tree.

3.2.4 A basic index construction algorithm

The TSF-index can be built by computing the TSFs for all the possible time windows and compressing them compactly. We show the steps in Algorithm 4. We first anchor each start time t_i (line 1). Then for each $t_j \in [t_i, t_{max}]$, we initialize $\Psi_{[t_i,t_j]}$ by $\Psi_{[t_i,t_j-1]}$ (lines 2-4). Afterwards, we process the edges in $[t_i, t_{max}]$ in chronological order; that is, for each edge (u, v, t_j), if vertices u and v are not connected via the edges in $\Psi_{[t_i,t_j]}$, we add it to $\Psi_{[t_i,t_j]}$ (lines 5-7). In the meantime, if the edge is added for the first time, we put it into $F(t_i, t_j)$ according to Lemma 3.6 (lines 8-9). After processing every

time interval $[t_i, t_j]$, we can derive the set of edges in each $F(t_i, t_j)$ correspondingly. Lastly, we insert each $F(t_i, t_j)$ to its corresponding $F_p[\,]$ and $F_b[\,]$ (lines 10-12).

Algorithm 4: TSF-construct(G)

```
Input: an undirected temporal graph G = (V, E);
Output: the corresponding TSF-index;
1  for t_i ← 0, ··· , t_max do
2      for t_j ← t_i, t_i + 1, ··· , t_max do
3          Ψ_[t_i,t_j] ← ∅;
4          if t_j > t_i then Ψ_[t_i,t_j] ← Ψ_[t_i,t_j−1];
5          for (u, v, t_j) ∈ E do
6              if u and v are not connected by Ψ_[t_i,t_j] then
7                  Ψ_[t_i,t_j] ← Ψ_[t_i,t_j] ∪ {(u, v, t_j)};
8                  if (u, v, t_j) ∉ Ψ_[t_i−1,t_max] then
9                      F(t_i, t_j) ← F(t_i, t_j) ∪ {(u, v, t_j)};
10 for t_i ← 0, ··· , t_max do
11     for t_j ← t_i, t_i + 1, ··· , t_max ∧ F(t_i, t_j) ≠ ∅ do
12         append F(t_i, t_j) to F_p[t_j] and F_b[⌊t_j/√t_max⌋];
```

Lemma 3.9 *Algorithm 4 costs $O(mt_{max})$ time.*

3.2.5 An optimized method of index construction

By Lemma 3.6, we know that given a temporal edge (u, v, t), its appearing time interval must be $[\widehat{t}, t]$ for some $\widehat{t} \in [0, t]$. Therefore, we can alternatively construct the TSF-index by determining $\widehat{t}$ of each edge. This can be done by enumerating the temporal edges chronologically and maintaining a maximum spanning forest (MSF) [63].

We first introduce the concept of MSF, which could be viewed as a special type of TSF.

Definition 3.4 (*MSF*) Given a temporal graph $G = (V, E)$ and a time window $[t_s, t_e]$, the MSF over $[t_s, t_e]$, annotated as $\Upsilon_{[t_s,t_e]}$, is the $\Psi_{[t_s,t_e]}$ with the largest sum of edge timestamps.

The main process of determining $\widehat{t}$ of all edges runs in three steps: First, initialize an empty Υ and enumerate all the edges chronologically. Then for each edge (u, v, t), determine $\widehat{t}$ of (u, v, t) by searching in Υ. Lastly, update Υ with the edge (u, v, t).

Lemma 3.10 *Given a newly inserted edge (u, v, t), with u and v already connected in the Υ. Let $\mathcal{P}_{uv}$ denote the unique simple path connecting u and v in the current Υ. By the lemma statement, let $e_{min} = (x, y, t_{min})$ be the edge on $\mathcal{P}_{uv}$ with the minimum timestamp. Then the $\widehat{t}$ of (u, v, t) is $t_{min}+1$,*

Step 1 is straightforward. In steps 2 and 3, our core idea is to maintain $\Upsilon_{[0,t]}$ as t increases. To be more specific, we can determine $\widehat{t}$ and update Υ by considering the following two different cases:

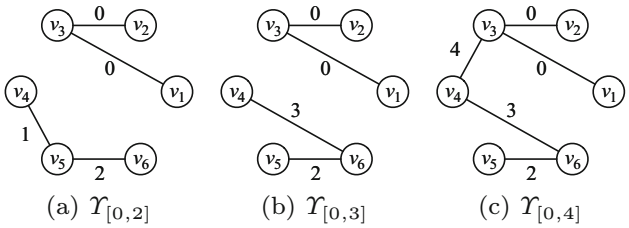


Fig. 4 MSF of the temporal graph in Figure 1(a)

1. u **and** v **are connected in** Υ. Based on Lemma 3.10, the appearing time interval of (u, v, t) is $[t_{min} + 1, t]$. We update Υ by deleting (x, y, t_{min}) from Υ and inserting (u, v, t) into Υ.
2. u **and** v **are not connected in** Υ. Since all the previous edges do not connect u and v, the appearing time interval of (u, v, t) is $[0, t]$, i.e., $\widehat{t} = 0$. We update Υ by inserting (u, v, t) into Υ.

Example 3.3 Given the temporal graph G shown in Figure 1(a) , Figure 4 depicts a partial process of maintaining the MSF of G. Consider the current $\Upsilon = \Upsilon_{[0,2]}$ as shown in Figure 4(a) and the current edge $(v_4, v_6, 3)$. Since v_4 and v_6 are connected in $\Upsilon_{[0,2]}$, we retrieve the path from v_4 to v_6 and delete the edge with the minimum timestamp on the path, which is $(v_4, v_5, 1)$. Therefore, the appearing time interval of $(v_4, v_6, 3)$ is $[4, 5]$, and we insert $(v_4, v_6, 3)$ into Υ to obtain $\Upsilon_{[0,3]}$ as shown in Figure 4(b) . The next edge is $(v_3, v_4, 4)$. Since v_3 and v_4 are not connected in the $\Upsilon_{[0,3]}$, the appearing time interval of $(v_3, v_4, 4)$ is $[0, 4]$, and we insert $(v_3, v_4, 4)$ into Υ to obtain $\Upsilon_{[0,4]}$ as shown in Figure 4(c) .

Algorithm 5 details the process of constructing the TSF-index based on maintaining an MSF. We first initialize an empty MSF and TSF-index (line 1). Then we enumerate the temporal edges chronologically to maintain the MSF and construct the TSF-index (lines 2-10). For case 1, we retrieve the edge (x, y, t_{min}), update the MSF and insert it into $F(t_{min} + 1, t)$ (lines 3-7). For case 2, we insert (u, v, t) into $F(0, t)$ (lines 8-9). We then insert the (u, v, t) into Υ (line 10). Lastly, we insert each $F(t_i, t_j)$ to its corresponding $F_p[]$ and $F_b[]$ (lines 11-13).

Lemma 3.11 *Given the time complexity of updating the MSF with one edge is $O(\tau)$, the time complexity of Algorithm 5 is $O(m\tau)$.*

As shown in Algorithm 5, updating the MSF with one edge requires retrieving P, deleting (x, y, t_{min}) from Υ and inserting (u, v, t) into Υ. A straightforward implementation is to perform a traversal algorithm from u to find P, which costs $O(n)$ time because $|\Upsilon| = O(n)$. Therefore, applying this traversal-based update over m insertions leads to an $O(mn)$ construction time in the worst case,

Algorithm 5: MSF-construct(G)

Input: an undirected temporal $G = (V, E)$;
Output: the corresponding TSF-index;
1 $\Upsilon, F \leftarrow \emptyset$;
2 **foreach** $(u, v, t) \in E$ *in chronological order* **do**
3 **if** *u and v are connected in* Υ **then**
4 $P \leftarrow$ the path connecting u, v in Υ;
5 $(x, y, t_{min}) \leftarrow$ the edge with the minimum timestamp in P;
6 $\Upsilon \leftarrow \Upsilon \setminus \{(x, y, t_{min})\}$;
7 $F(t_{min} + 1, t) \leftarrow F(t_{min} + 1, t) \cup \{(u, v, t)\}$;
8 **else**
9 $F(0, t) \leftarrow F(0, t) \cup \{(u, v, t)\}$;
10 $\Upsilon \leftarrow \Upsilon \cup \{(u, v, t)\}$;
11 **for** $t_i \leftarrow 0, \cdots, t_{max}$ **do**
12 **for** $t_j \leftarrow t_i, t_i + 1, \cdots, t_{max} \wedge F(t_i, t_j) \neq \emptyset$ **do**
13 append $F(t_i, t_j)$ to $F_p[t_j]$ and $F_b[\lfloor \frac{t_j}{\sqrt{t_{max}}} \rfloor]$;

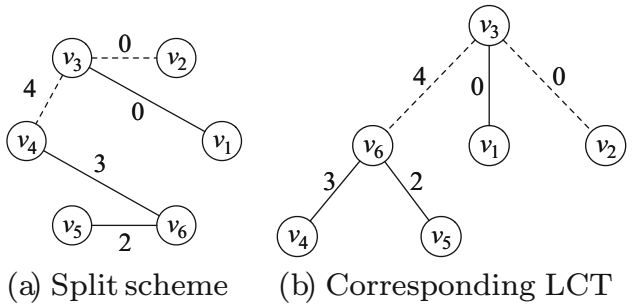


Fig. 5 The LCT of $\Upsilon_{[0,4]}$ in Figure 4(c)

which does not scale to large graphs. We implemented this partial-recomputation baseline and report its construction time as `MSF-construct-brute` in Figure 12(a) . As shown in the results, this brute force algorithm degrades significantly on large datasets, and is consistently slower than both the TSF-index construction and our LCT-based `MSF-construct`. Nevertheless, several data structures can accelerate the process of updating the MSF, including the Link-Cut Tree (LCT) [61], the PEC-index [63], and the topology tree [25]. Among these, LCT excels in both theoretical complexity and practical performance. By implementing the LCT with splay trees [16], we can update the MSF within $O(\log n)$ time.

Here, we offer an overview of the LCT implemented with splay trees. In LCT, the MSF is divided into several rooted trees and each rooted tree is divided into several chains. Each splay tree corresponds to a chain where the in-order traversal sequence [40] of a splay tree corresponds to the result of sorting the vertices on the chain in ascending order by their distance from the root. Figure 5 shows a chain-split scheme and the corresponding LCT of the $\Upsilon_{[0,4]}$ in Figure 4(c) .

Example 3.4 Given the $\Upsilon_{[0,4]}$ in Figure 4(c) , we can split it into three chains as shown in Figure 5(a) : $\{v_1, v_3\}$, $\{v_2\}$, and $\{v_4, v_5, v_6\}$. Solid lines represent the edges within these chains, while dashed lines represent the edges outside the chains. Since $\Upsilon_{[0,4]}$ is connected, we assign v_3 as the root of the $\Upsilon_{[0,4]}$. We generate a splay tree for each chain as shown in Figure 5(b) . For instance, the in-order traversal sequence of the splay tree corresponding to the chain $\{v_4, v_5, v_6\}$ is $v_4 \rightarrow v_6 \rightarrow v_5$, which matches the result of sorting the vertices on the chain in ascending order by their distance from the root.

As shown in [61], it takes $O(\log n)$ time to update the MSF with one edge in the LCT. Thus, $\tau = \log n$ and Algorithm 5 costs $O(m \log n)$ time by Lemma 3.11.

3.2.6 Index maintenance algorithms

When a new edge $(u, v, t_{max} + 1)$ is inserted into the graph, we can update the TSF-index based on Algorithms 4 and 5 respectively. For Algorithm 4, we can revise it such that t_i is ranged from 0 to $(t_{max}+1)$ in the outer loops (lines 1 and 10). When $t_i \in [0, t_{max}]$, the variable t_j in the inner loop is fixed as $(t_{max} + 1)$ (lines 2 and 11), and instead of initializing the TSF by lines 3-4, we reconstruct $\Psi_{[t_i, t_{max}]}$ from the current TSF-index and then run the steps in lines 5-9 and 12; when $t_i = t_{max} + 1$, we run the original procedure of the inner loop.

We can also maintain the TSF-index using Algorithm 5. Given a newly inserted edge $(u, v, t_{max} + 1)$, we use lines 3-10 in Algorithm 5 since this part updates the MSF and the edge set F with exactly one edge. Afterwards, we can update the TSF-index based on the new edge set F. Specifically, the updated edge set $F(t_i, t_j)$ must have $t_j = t_{max} + 1$, and we can append it into $F_p[t_{max} + 1]$ and $F_b[\lfloor \frac{t_{max}+1}{\sqrt{t_{max}}} \rfloor]$.

Lemma 3.12 *The time cost of maintaining the TSF-index based on Algorithm 5 is $O(\log n)$, independent of the t_{max} and m.*

Our later experiments show that Algorithm 5 and the corresponding index maintenance algorithm outperform the basic index construction and maintenance algorithms, respectively.

4 Index-based solutions for directed temporal graphs

Inspired by the indices for undirected graphs, we first give a spanning forest-based index for window-SCC queries on directed temporal graphs. To further reduce the indexing time and space cost, we propose an advanced index with $O(m)$ space.

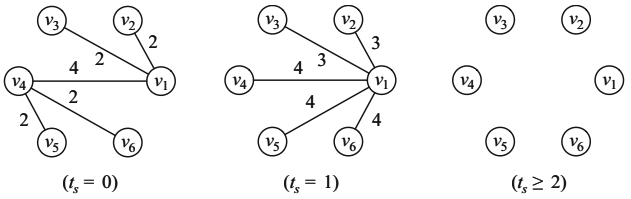


Fig. 6 Constructed forests with different $t_s \in [0, 4]$

4.1 A baseline index-based solution

4.1.1 Index overview

In this section, we propose an index named D-baseline index. Unlike undirected graphs, an SCC in the directed graph cannot be represented by a directed spanning tree [27], so we cannot build a directed version of the TSF-index for directed temporal graphs. Instead, we propose another forest structure of undirected edges, which connects the vertices in each SCC. Specifically, we maintain an undirected edge set $D(t_s)$ of the forest for each start time t_s, where edges are associated with timestamps. If u and v are in the same SCC in $G_{[t_s,t_e]}$, then u and v are connected via the edges in $D(t_s)$ with timestamps $\leq t_e$. Figure 6 depicts the constructed forests for the graph in Figure 1(b) .

Lemma 4.1 *The D-baseline index costs $O(nt_{max})$ space.*

4.1.2 Query processing

To solve the query over a time window $[t_s, t_e]$, we first retrieve all the edges in $D(t_s)$ with timestamps $\leq t_e$. Then, we construct a new static graph $G' = (V, E')$ where V contains all the vertices of the graph G and E' represents the retrieved edges. Finally, we detect all the CCs from graph G'. Algorithm 6 shows the steps above. It firstly initializes a graph G' without edges (line 1), then adds all the edges in $D(t_s)$ with timestamps in $[t_s, t_e]$ into G' (line 2), and finally returns CCs on G' as the window-SCCs (line 3), which actually are the same as the SCCs in $G_{[t_s,t_e]}$.

Algorithm 6: D-baseline-query(D, $[t_s, t_e]$)

Input: the index D and the query time window $[t_s, t_e]$;
Output: all window-SCCs in $G_{[t_s,t_e]}$;
1 $G' \leftarrow (V, E')$ where $E' = \emptyset$;
2 **for** $e = (u, v, t) \in D(t_s) \wedge t \leq t_e$ **do** $E' \leftarrow E' \cup \{(u, v)\}$;
3 **return** all the CCs computed from G';

Lemma 4.2 *Algorithm 6 answers a query in $O(n)$ time.*

4.1.3 Index construction

To build the index, for each start time t_s, we iteratively find all SCCs in $G_{[t_s,t]}$ for each $t \in [t_s, t_{max}]$, and create the minimal undirected edges to connect the vertices newly involved in the same SCC at time t. Algorithm 7 gives the details. We first anchor each start time t_s and initialize $D(t_s)$ (lines 1-2). As the time window expands to $[t_s, t]$, we run algorithms to find all SCCs in $G_{[t_s,t]}$ (lines 3-4). Then for each SCC, we add undirected edges into D until the vertices in the SCC are connected by the edges in D (lines 5-9).

Algorithm 7: D-baseline-construct(G)

Input: a directed temporal graph $G = (V, E)$;
Output: the index $D(t_s)$ for all $0 \leq t_s \leq t_{max}$;
1 **for** $t_s = 0, 1, \cdots, t_{max}$ **do**
2 $D(t_s) \leftarrow \emptyset$;
3 **for** $t \leftarrow t_s, t_s + 1, \cdots, t_{max}$ **do**
4 run algorithms to find SCCs on $G_{[t_s,t]}$;
5 **for** *each SCC in* $G_{[t_s,t]}$ **do**
6 choose a vertex u in the SCC;
7 **for** *all other vertices v in the SCC* **do**
8 **if** *u and v are not connected by* $D(t_s)$ **then**
9 $D(t_s) \leftarrow D(t_s) \cup \{(u, v, t)\}$;

Lemma 4.3 *Algorithm 7 costs $O((m + n)t_{max}^2)$ time.*

In addition, when a new edge $(u, v, t_{max} + 1)$ arrives, we can update the D-baseline index without re-building it from scratch. Specifically, we can design a revised version of Algorithm 7, by ranging t_s from 0 to $(t_{max} + 1)$ in its outer loop. When $t_s \in [0, t_{max}]$, the variable t in the inner loop (line 3) is fixed as $t = t_{max} + 1$; when $t_s = t_{max} + 1$, we run the original procedure of the inner loop. The maintenance cost per new edge is $O((m+n)t_{max})$, since the algorithm iterates over $O(t_{max})$ start times, and for each t_s, processing the additional edge to run SCC algorithm on $G_{[t_s,t_{max}+1]}$ requires at most $O(m + n)$ time.

4.2 An advanced index-based solution

The solution based on the D-baseline index handles small graphs smoothly but is costly when t_{max} is large. To reduce the enormous indexing cost caused by t_{max}, we propose an advanced index, called reconstruction edge set (RES) index, or RES-index, by maintaining small sets of edges to reconstruct SCCs, whose space cost is $O(m)$.

4.2.1 The concept of RES and its properties

The D-baseline index does not have the nested relationships w.r.t. different anchoring start times, as depicted in Figure

6. Hence, it is hard to design a space-efficient index by compressing the D-baseline indexes. Inspired by TSF-index (Definition 3.3), we can also extract the directed edge subset of E to reconstruct the SCCs concerning different time windows. We conjecture that such reconstruction edge sets can have nested relationships that allow compression. To verify it, we determine the edges contributing to SCCs by defining the reconstruction edge set (RES) in Definition 4.1.

Definition 4.1 (*RES*) Given a directed temporal graph $G = (V, E)$ and a time window $[t_s, t_e]$, the RES over $[t_s, t_e]$ is an edge set $\Phi_{[t_s,t_e]} \subseteq E$, such that all the SCCs of the graph $G' = (V, \Phi_{[t_s,t_e]})$ are exactly the window-SCCs of $G_{[t_s,t_e]}$.

As the RES's are designed for indexing, it is desired that the size of $\Phi_{[t_s,t_e]}$ can be bounded by $O(n)$. To fulfill such a requirement, we give an approach to find $\Phi_{[t_s,t_e]}$ by carefully choosing edges that strongly connect an SCC. Inspired by the ideas of Kosaraju algorithm [60], we can perform two BFS procedures on an SCC with k vertices and its reverse graph, respectively, and use the edges gone through by the procedures to represent the SCC, where the number of such edges is at most $2k - 2$. Iterating over all SCC's, we can obtain $\Phi_{[t_s,t_e]}$, which has most $2n - 2$ edges. The details to process an SCC are presented in Algorithm 8. We first obtain the reverse graph of G' and then select a vertex in S randomly (lines 1-2). Next, we use BFS to find the edges in G' and the reverse of G' (lines 3-6).

Algorithm 8: Find-RES(G', S)

Input: an SCC S and the subgraph G' induced by S;
Output: a set of no more than $2|S| - 2$ edges representing S;
1 $\overline{G}' \leftarrow$ the reverse graph of G';
2 select a vertex $u \in S$ randomly;
3 perform BFS on S in G' starting from u;
4 $E_1 \leftarrow$ the edge set which BFS went through;
5 perform BFS on S in $\overline{G}'$ starting from u;
6 $E_2 \leftarrow$ the edge set which BFS went through;
7 $\overline{E}_2 \leftarrow$ the reverse edges in E_2;
8 **return** $E_1 \cup \overline{E}_2$;

Similar to TSFs, there exist two kinds of overlapping relationships among different RES's, discussed as follows.

- **Nested relationship by anchoring a start time.** We formalize this relationship by Lemma 4.4.

Lemma 4.4 *Given a directed temporal graph G and an anchored start time t_s, there exists a chain of RES's such that:*

$$\Phi_{[t_s,t_s]} \subseteq \Phi_{[t_s,t_s+1]} \subseteq \cdots \subseteq \Phi_{[t_s,t_{max}]}$$

By Lemma 4.4, for each anchored start time t_s, we can compress all RES's $\Phi_{[t_s,t_s]}$, $\Phi_{[t_s,t_s+1]}$, $\cdots$, $\Phi_{[t_s,t_{max}]}$ using $O(n)$ space cost, by only keeping the newly added edges for each RES with the help of Algorithm 8. For example, compressing the RES's for $t_s = 0$ will give us $\Phi_{[0,t_{max}]}$ in the leftmost subfigure of Figure 7. Taking a closer look at Figure 7, we can find that filtering the edges appearing in [0, 1] in the RES $\Phi_{[0,t_{max}]}$ gives us more edges than $\Phi_{[0,1]}$, which is different from TSFs. A natural question is whether those edges $\Phi_{[0,t_{max}]} \cap E_{[0,1]}$ connect the same SCCs with $\Phi_{[0,1]}$. We prove that $\Phi_{[t_s,t_{max}]} \cap E_{[t_s,t_e]}$ is indeed a valid RES to recover the corresponding SCCs by Lemmas 4.5 and 4.6.

Lemma 4.5 *If $E_1 \subseteq E_{[t_s,t_e]}$ is a valid RES for some $\Phi_{[t_s,t_e]}$, then $E_1 \cup E_2$ is also a valid RES for $\Phi_{[t_s,t_e]}$, where $E_2 \subseteq E_{[t_s,t_e]}$.*

Lemma 4.6 *$\Phi_{[t_s,t_{max}]} \cap E_{[t_s,t_e]}$ is a valid RES for $\Phi_{[t_s,t_e]}$.*

By Lemma 4.6, it is safe to only keep track of $\Phi_{[t_s,t_{max}]}$ for each $t_s \in [0, t_{max}]$. However, if we only compress RES's by each anchored start time, the index covering all start times will still take $O(n \cdot t_{max})$ space cost, which is the same as that of D-baseline index. To alleviate the enormous space cost caused by t_{max}, we exploit another overlapping relationship among RES's.

- **Overlapping relationship for multiple start times.** We present an important observation in Lemma 4.7, which allows us to compress all the RES's by using only $O(m)$ space.

Lemma 4.7 *Given an directed temporal graph G, there exist $(t_{max} + 1)$ RES's, i.e., $\Phi_{[0,t_{max}]}$, $\Phi_{[1,t_{max}]}$, $\cdots$, $\Phi_{[t_{max},t_{max}]}$, such that for each edge (u, v, t), there is a chain for some $t_l, t_r \in [0, t]$:*

$$\Phi_{[t_l,t_{max}]}, \Phi_{[t_l+1,t_{max}]}, \cdots, \Phi_{[t_r,t_{max}]}$$

satisfying (1) each RES in the chain contains the edge and (2) the edge is excluded in any RES's that are not in the chain.

By Lemma 4.7, for each edge (u, v, t), we can identify an *appearing time interval* $[t_l, t_r]$ such that for any $t' \in [t_l, t_r]$, the RES over $[t', t_{max}]$ contains it. For example, the edge $(v_5, v_6, 1)$ appears in the RES's starting from $t_s \in [0, 1]$, as shown in Figure 7.

4.2.2 Index overview

After obtaining the appearing time intervals for all edges, we can reversely organize the edges and their appearing time intervals, such that for each time interval $[t_i, t_j]$ where $0 \leq t_i \leq t_j \leq t_{max}$, there is a set of edges corresponding to

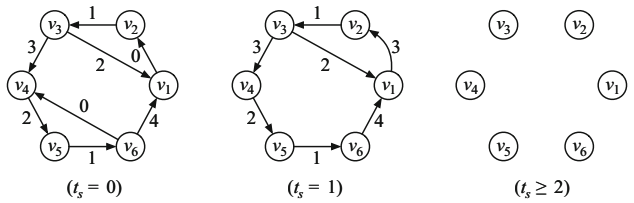


Fig. 7 The structure of RES's

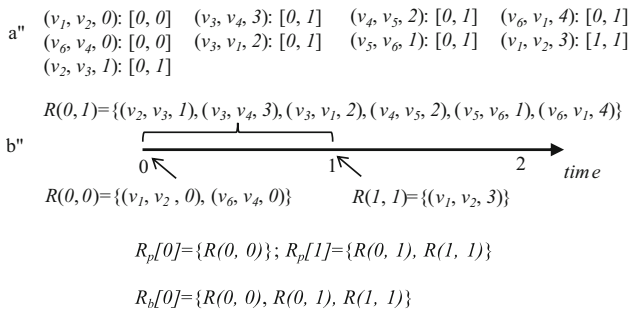


Fig. 8 The RES-index for the graph in Figure 1(b)

it, denoted by $R(t_i, t_j)$. To reduce the time complexity, the RES-index then organizes all non-empty $R(t_i, t_j)$ using two complementary data structures similar to the TSF-index:

1. **Point array** ($R_p[\,]$): For each timestamp $x \in [0, t_{max}]$, $R_p[x]$ stores all edge sets $R(t_i, t_j)$ with $t_j = x$.
2. **Block array** ($R_b[\,]$): For each integer $y \in [0, \lfloor\sqrt{t_{max}}\rfloor]$, $R_b[y]$ stores $R(t_i, t_j)$ with $t_j \in [y\sqrt{t_{max}}, (y+1)\sqrt{t_{max}})$.

Within both arrays, edge sets are maintained in ascending order of t_i to facilitate efficient retrieval.

Example 4.1 Figure 8(a) presents the appearing time interval for each edge of the graph in Figure 1(b) . Based on the appearing time intervals, Figure 8(b) presents the edge sets R and the arrays $R_p[\,]$ and $R_b[\,]$ in RES-index. Specifically, since $\sqrt{t_{max}} = 2$, $R_b[0]$ stores $R(t_i, t_j)$ with $t_j \in [0, 2)$.

Lemma 4.8 *The RES-index costs $O(m)$ space.*

4.2.3 Query processing

To answer the query over a time window $[t_s, t_e]$, we first utilize the RES-index to find all the edges in the RES over $[t_s, t_e]$. Then we can construct a new static graph $G' = (V, E')$ where V remains the same as the original graph G and E' represents the retrieved edges.

Algorithm 9 presents the details, which is very similar to Algorithm 3. The algorithm first constructs an empty graph G' (line 1). Then, it adds all the edges in $R(t_i, t_j)$ where $t_i \leq t_s$ and $t_s \leq t_j \leq t_e$ into G' (lines 2-12). Specifically, considering $l = \lfloor\frac{t_s}{\sqrt{t_{max}}}\rfloor + 1$ and $r = \lfloor\frac{t_e}{\sqrt{t_{max}}}\rfloor$, we divide the target edge sets $R(t_i, t_j)$ with $t_i \leq t_s$ and $t_s \leq t_j \leq t_e$ into three parts:

1. $R(t_i, t_j)$ with $t_i \leq t_s$ and $t_j \in [t_s, l\sqrt{t_{max}})$.
2. $R(t_i, t_j)$ with $t_i \leq t_s$ and $t_j \in [l\sqrt{t_{max}}, r\sqrt{t_{max}})$.
3. $R(t_i, t_j)$ with $t_i \leq t_s$ and $t_j \in [r\sqrt{t_{max}}, t_e]$.

We then fetch $R(t_i, t_j)$ in the first and third parts from $R_p[\,]$ (lines 4-6, 10-12) and $R(t_i, t_j)$ in the second part from $R_b[\,]$ (lines 7-9). Finally, we run SCC algorithms on the graph G' to return all the SCCs (line 13).

Algorithm 9: RES-query($R_p[\,]$, $R_b[\,]$, t_s, t_e)

Input: the RES-index $R_p[\,]$ and $R_b[\,]$, and query time window $[t_s, t_e]$;
Output: all window-SCCs in $G_{[t_s,t_e]}$;

```
1  G' ← (V, E'), E' ← ∅;
2  l ← ⌊ts/√tmax⌋ + 1;
3  r ← ⌊te/√tmax⌋;
4  for tj ∈ [ts, l√tmax) do
5      for R(ti, tj) ∈ Rp[tj] ∧ ti ∈ [0, ts] do
6          E' ← E' ∪ R(ti, tj);
7  for x ∈ [l, r) do
8      for R(ti, tj) ∈ Rb[x] ∧ ti ∈ [0, ts] do
9          E' ← E' ∪ R(ti, tj);
10 for tj ∈ [r√tmax, te] do
11     for R(ti, tj) ∈ Rp[tj] ∧ ti ∈ [0, ts] do
12         E' ← E' ∪ R(ti, tj);
13 return all the SCCs computed from G';
```

Lemma 4.9 *Algorithm 9 answers a query in $O(n + \sqrt{t_{max}})$ time.*

4.2.4 A two-pointer optimization

As mentioned above, we would need to compute $\Phi_{[t_s,t_{max}]}$ for each $t_s \in [0, t_{max}]$ before obtaining RES-index. A straightforward idea is to run SCC algorithms on $G_{[t_s,t_e]}$ for all time windows. This is unrealistic since it costs $O((m+n)t_{max}^2)$ time. Fortunately, we observe that for each edge with the start time $t_s \in [t_l, t_r]$ in Lemma 4.7, the smallest t when it is involved in an SCC of $G_{[t_s,t]}$ is increasing as t_s increases. We define such t as the **effective time** in Definition 4.2 and we formally prove the observation above in Lemma 4.10.

Definition 4.2 (*Effective time*) Given an edge e and a start time t_s, if both vertices of e are in the same SCC of $G_{[t_s,t]}$, but they are not in the same SCC of $G_{[t_s,t-1]}$, then the effective time of e at t_s is defined as $T(t_s, e) = t$. If e is never involved in an SCC, $T(t_s, e) = t_{max} + 1$.

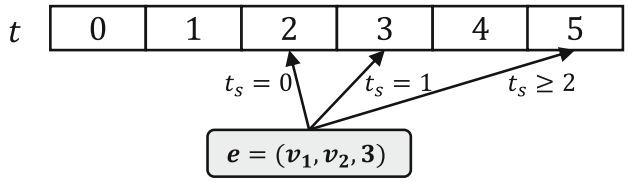


Fig. 9 Effective times of $e = (v_1, v_2, 3)$

Lemma 4.10 (Monotonicity of effective time) *For an edge e in G, it holds for all $t_s \geq 1$ that $T(t_s, e) \geq T(t_s - 1, e)$.*

By Lemma 4.10, the effective times of an edge e at different t_s from its appearing time interval $[t_l, t_r]$ are monotonically increasing.

Example 4.2 As shown in Figure 9, the edge $e = (v_1, v_2, 3)$ has effective times 2, 3, 5. When $t_s = 0$, v_1, v_2 starts to form an SCC at time $T(0, e) = 2$. Similarly $T(1, e) = 3$. When $t_s \geq 2$, v_1, v_2 can never form an SCC. Then $T(t_s, e) = t_{max} + 1 = 5$ for $t_s \geq 2$. Hence, this edge e only needs to be considered when running SCC algorithms for $G_{[t_s, t']}$, where $T(t_s - 1, e) \leq t' \leq T(t_s, e)$.

The number of different time windows where an edge is considered is $O(t_{max})$ by Lemma 4.11. Besides, we do not need to iterate all the vertices for each time window when running SCC algorithms. The vertices disconnected from other vertices can be skipped. Since the number of vertices connected to other vertices is at most twice the number of edges, the overall time cost of iterating the vertices is also bounded by $O(mt_{max})$. Therefore, the total time complexity of constructing the RES-index is reduced to $O(mt_{max})$.

Lemma 4.11 *For an edge e in G, it can be considered in $2t_{max} + 1 = O(t_{max})$ different time intervals.*

4.2.5 Index construction

Based on the above two-pointer optimization, the RES-index can be built by computing the RES's for all possible time windows efficiently and compressing them compactly. Algorithm 10 shows the detailed steps. We first anchor each start time t_i and initialize G' (lines 1-2). As the time window expands to $[t_i, t_j]$, we initialize $\Phi_{[t_i, t_j]}$ by $\Phi_{[t_i, t_j - 1]}$ based on Lemma 4.4 (lines 3-5).

For each original edge $e = (u, v, t_j) \in E_{t_j}$, we map it into G' by adding (u', v') into E' and adding u' and v' into V' (lines 6-9). We clear E_{t_j} and run algorithms to find SCCs on G' (line 10). For each SCC found (line 11), we iterate the internal edges and put the corresponding original edges into E_{t_j} (lines 12-13). Hence, for the next anchored start time $t_i + 1$, those edges will only be accessed at timestamp t_j, i.e., the effective time at t_i. Afterward, we find the RES edges in E' which can connect all pairs of vertices in the SCC with bidirectional paths via Algorithm 8 (line 14). We add the corresponding original edges of them into $\Phi_{[t_i, t_j]}$ and shrink the SCC in G' into a new vertex (line 15). After processing every $t_j \in [t_i, t_{max}]$, we can derive $\Phi_{[t_i, t_{max}]}$ and the corresponding edges in R (lines 16-17).

Lastly, we insert each $R(t_i, t_j)$ to its corresponding $R_p[\,]$ and $R_b[\,]$ (lines 18-20).

Algorithm 10: RES-construct(G)

```
Input: a directed temporal graph G = (V, E);
Output: the corresponding RES-index;
1  for t_i ← 0, ··· , t_max do
2      G' ← (V', E') where V' ← ∅, E' ← ∅;
3      for t_j ← t_i, ··· , t_max do
4          Φ_[t_i,t_j] ← ∅;
5          if t_j > t_i then Φ_[t_i,t_j] ← Φ_[t_i,t_j−1];
6          for e = (u, v, t_j) ∈ E_{t_j} do
7              Map u, v to u', v';
8              E' ← E' ∪ {e' = (u', v')};
9              V' ← V' ∪ {u', v'};
10         Clear E_{t_j} and run algorithms to find SCCs on G';
11         for each SCC in G' do
12             for each internal edge e' in the SCC do
13                 e ← the original edge of e'; Add e into E_{t_j};
14             Find RES edges E_S for the SCC by calling Algorithm
                   8;
15             Add E_S to Φ_[t_i,t_j] and shrink the SCC into a vertex;
16     for e ∈ Φ_[t_i,t_max] \ Φ_[t_i−1,t_max] do t_l(e) ← t_i;
17     for e ∈ Φ_[t_i−1,t_max] \ Φ_[t_i,t_max] do add e into R(t_l(e), t_i − 1);
18 for t_i ← 0, ··· , t_max do
19     for t_j ← t_i, t_i + 1, ··· , t_max ∧ R(t_i, t_j) ≠ ∅ do
20         append R(t_i, t_j) to R_p[t_j] and R_b[⌊t_j / √t_max⌋];
```

Lemma 4.12 *Algorithm 10 costs $O(mt_{max})$ time.*

4.2.6 Index maintenance algorithm

When a new edge $e_{new} = (u, v, t_{max} + 1)$ is inserted into the graph, the RES-index can be efficiently maintained without recomputing from scratch. Specifically, we can design the maintenance algorithm based on Algorithm 10. In the outer loop (line 1), we range variable t_i from 0 to $(t_{max} + 1)$. In the inner loop, when $t_i \in [0, t_{max}]$, the variable t_j is fixed as $(t_{max} + 1)$, and instead of initializing the RES by lines 4-5, we reconstruct $\Phi_{[t_i, t_{max}]}$ from the current RES-index, initialize E' as $E_{[t_i, t_{max}]}$, and run the steps in lines 6-17; when $t_i = t_{max} + 1$, we follow the original procedure. Last, we insert the updated $R(t_i, t_j)$ into the corresponding $R_p[t_j]$ and $R_b[\lfloor \frac{t_j}{\sqrt{t_{max}}} \rfloor]$. The time complexity of this RES-index update algorithm is $O(mt_{max})$. Although this complexity class is the same as the initial construction, the incremental operation

is practically faster for a small number of edge insertions because it skips the initialization and verification steps for $t_j \leq t_{max}$. However, for large-batch updates, the cumulative cost of incremental maintenance will eventually surpass the cost of rebuilding the index from scratch. As shown in Table 11, there exists a clear break-even threshold (e.g., 8.2K insertions for the EE dataset) where the strategy should switch from incremental updates to full reconstruction to maintain optimal performance.

Moreover, achieving a faster update algorithm is theoretically challenging because inserting a single edge into a directed graph can trigger cascading structural updates, bringing numerous previously excluded historical edges into the RES-index. Identifying and updating these dormant edges to maintain index completeness requires costly graph traversals, often approaching the expense of full reconstruction.

5 Discussions

5.1 Extension to interval-based temporal graph model

In our problem setting, each edge $e = (u, v, t)$ is represented as a triplet, indicating that an edge from u to v appears at a discrete timestamp t. An alternative definition of a temporal graph associates each edge with a time interval $[t_s, t_e]$, meaning that the edge exists continuously from timestamp t_s to t_e [11, 38].

TSF-index under the interval-based model. We extend the TSF-index to temporal edges with active intervals without increasing the asymptotic construction cost. In the interval-based model, each edge is represented as $(u, v, [t_s, t_e])$, meaning that it is present in every snapshot G_t for $t \in [t_s, t_e]$. To construct the index, we sort all edges in non-decreasing order of their start time t_s and process them in this order. When an edge $(u, v, [t_s, t_e])$ is processed at time t_s, we insert it into the maintained MSF using its end time t_e as the edge timestamp. Intuitively, using t_e makes the MSF maintain the edge until it is replaced by edges with end time larger than t_e, which is exactly what is needed when answering connectivity queries over a future time interval. Algorithm 5 can then be adapted by simply replacing the timestamp t of a discrete edge (u, v, t) with t_e for an interval edge $(u, v, [t_s, t_e])$ (while still processing edges by increasing t_s). All MSF updates (finding the path in the current MSF, comparing the minimum weight on the path with t_e, and performing replacement if needed) remain unchanged. Therefore, the construction complexity is the same as in the discrete-time setting. For answering a query interval $[t_a, t_b]$, we retrieve the same candidate TSF as in the discrete case, and then filter the returned edges by keeping only those whose active interval intersects $[t_a, t_b]$ (i.e., $[t_a, t_b] \cap [t_s, t_e] \neq \emptyset$). This post-filtering step does not change the asymptotic query cost since the TSF size remains $O(n)$.

We have proposed a detailed pseudo-code of the revised TSF-index construction and query algorithm in the supplementary information.

RES-index under the interval-based model. We extend the RES-index to the interval-based model where each edge is given as $(u, v, [t_s, t_e])$, meaning that it is active for all $t \in [t_s, t_e]$. A key observation is that if an interval edge never belongs to any SCC in $G_{[t,t_{\max}]}$ for all $t \geq t_s$, then it will never contribute to any RES set $\Phi_{[t,t_{\max}]}$. Such an edge is therefore equivalent to a discrete-time edge (u, v, t_s) in our construction, and does not require any special handling.

For the remaining edges that may affect SCCs beyond their appearing time, the extension is conceptually simple: we still process edges in non-decreasing order of t_s, but an edge may remain *effective* across multiple subsequent timestamps as long as its interval covers them. Hence, an interval edge is carried forward only while it is still active (i.e., as long as $t \leq t_e$), and once it can no longer influence future SCCs, it is naturally discarded. This ensures that we do not introduce additional asymptotic overhead compared to the original RES-index construction. For details, we have also provided a pseudo-code of the revised RES-index construction algorithm in the supplementary information.

Query processing of the RES extension is identical to the TSF extension: after retrieving candidate RES edges from the index, we keep only those with intervals intersecting the query window $[t_a, t_b]$ (i.e., $t_s \leq t_b$ and $t_e \geq t_a$), which does not change the asymptotic query time.

5.2 Extension for edge deletions

Our proposed maintenance algorithms focus on edge insertions with new timestamps, which represent the most common update pattern in temporal graphs, as new information typically arrives with the most recent timestamps. Although edge deletions may also occur (e.g., due to erroneous or retracted information), such operations are usually infrequent, and thus higher update costs are acceptable. In this subsection, we briefly outline how edge deletions can be supported. Given a deleted edge with timestamp t, our key idea is to identify a replacement edge within the interval $[t, t_{max}]$.

Deletion on TSF-index. To support edge deletions, we maintain the appearing time interval of each edge. Specifically, whenever an edge is inserted into $F(t_i, t_j)$ in line 7 or 9 of Algorithm 5, its appearing time interval is recorded as $[t_i, t_j]$. This means the edge appears in all TSFs $\Psi_{[t_s,t_{\max}]}$ for every $t_i \leq t_s \leq t_j$. Now consider deleting an edge $e = (u, v, t)$ whose appearing time interval is $[t_i, t_j]$. From lines 7 and 9 of Algorithm 5, we know that $t_j = t$. Intuitively, there is at most one edge in the timestamp range $[t, t_{\max}]$ that can

replace the deleted edge. We present this formally by the following lemma.

Lemma 5.1 *Let the originally indexed TSFs be Ψ. Upon deleting an edge $e = (u, v, t)$ whose appearing time interval is $[t_i, t]$, if u and v remain in the same window-CC in $[t, t_{\max}]$ after the deletion, then there exists a valid set of updated TSFs $\Psi'_{[t_s,t_{\max}]}$ and an edge e' with timestamp $\geq t$ such that:*

$$\Psi'_{[t_s,t_{\max}]} = \begin{cases} \Psi_{[t_s,t_{\max}]}, & 0 \leq t_s < t_i, \\ \Psi_{[t_s,t_{\max}]} \setminus \{e\} \cup \{e'\}, & t_i \leq t_s \leq t, \\ \Psi_{[t_s,t_{\max}]}, & t < t_s \leq t_{\max}. \end{cases}$$

Besides, there are only two possible cases of edges that can potentially serve as that replacement edge: (1) edges with timestamp t that were not previously added to the index, and (2) edges whose appearing time intervals start at $t + 1$. Edges with appearing time intervals starting at $t' > t + 1$ cannot be candidates, because such edges are absent from $\Psi_{[t+1,t_{\max}]}$. Moreover, by the monotonicity of Ψ with respect to the window start time, an edge cannot appear in $\Psi_{[t_s,t_{\max}]}$ for any $t_s \leq t$ while being absent from $\Psi_{[t+1,t_{\max}]}$. Hence, replacement candidates can only be edges with timestamp t that were not previously indexed, or edges whose appearing time intervals start at $t + 1$. To identify a replacement edge, we first reconstruct the window-CCs from $\Psi_{[t+1,t_{\max}]} \setminus \{e\}$ in $O(n)$ time. Next, we examine all edges with timestamp t to determine whether a replacement exists. If no suitable edge is found, we consider edges with appearing time intervals starting at $t + 1$, which appear in $\Psi_{[t+1,t_{\max}]}$ but not in $\Psi_{[t,t_{\max}]}$. The total number of such candidate edges is bounded by $O(n)$. We then iterate through these candidate edges in chronological order, and the first edge to connect two distinct window-CCs serves as the replacement edge. Consequently, the overall time complexity of the deletion procedure is $O(n + m_t)$, where m_t is the number of edges timestamped t. If no replacement edge is found, then deleting e splits the affected window-CC into two components for every $t_s \in [t_i, t]$; hence, we simply remove e from $\Psi_{[t_s,t_{\max}]}$ (without inserting any edge).

Deletion on RES-index. Edge deletions in the RES-index are more challenging than in the TSF-index, as there is no straightforward replacement edge in directed graphs. Deleting a directed edge may alter the structure of its corresponding SCC, potentially requiring a complete recomputation of the affected SCCs with different edge sets. Consequently, edge deletions on the RES-index require advanced techniques and are left for future work, and we simply reconstruct the RES-index if there is an edge deletion.

Although these costs are higher than insertion updates, they remain acceptable given that edge deletions are typically rare in most temporal graph applications.

5.3 Extension for timestamp coarsening approximation

While our proposed indices achieve linear space complexity $O(m)$, maintaining exact indices for graphs with billions of edges or extremely long time horizons can still be resource-intensive. To support such massive-scale scenarios, we propose a temporal coarsening approximation strategy that trades fine-grained temporal resolution for scalability improvements.

Specifically, we introduce a granularity parameter $\delta \geq 1$. We map the original timestamps to a coarser domain by aggregating every δ consecutive timestamps into a single logical timestamp. The mapping function for a timestamp t is defined as $T(t) = \lfloor t/\delta \rfloor$. Consequently, the original temporal graph $G = (V, E)$ is transformed into a coarsened graph $G' = (V, E')$, where the maximum timestamp is reduced to $t'_{max} = \lfloor t_{max}/\delta \rfloor$. In the following, we discuss two efficiency gains.

Reducing the number of edges m. By aggregating time, multiple edges with different timestamps (u, v, t_1), (u, v, t_2) that fall within the same bucket (i.e., $T(t_1) = T(t_2)$) are merged into a single edge in G'. This strategy reduces the total number of edges from m to m', where $m' \leq m$. Since the number of edges dominates the space complexity and construction time of both the TSF-index and RES-index, this directly reduces memory usage and index construction time.

Reducing the number of timestamps t_{max}. As aforementioned, by mapping the timestamps we reduce t_{max} to $t'_{max} = \lfloor t_{max}/\delta \rfloor$. Since the construction complexity of our RES-index is $O(mt_{max})$, the construction time is improved by a factor of roughly δ. Similarly, the query time complexity, which involves $\sqrt{t_{max}}$, is reduced to $O(n + \sqrt{t'_{max}})$.

5.4 Limitations of parallelized and distributed index

While parallel processing is a common optimization for large-scale graphs, extending our index construction algorithms to parallel settings faces significant theoretical challenges due to strong sequential dependencies. Specifically, the TSF-index construction relies on strictly chronological edge updates to maintain the Maximum Spanning Forest (MSF) via Link-Cut Trees (LCTs); parallelizing this breaks the necessary topological ordering, resulting in incorrectness. Similarly, the RES-index achieves its space complexity $O(m)$ by reusing connectivity states from previous time windows; parallel execution would break this dependency chain, degrading the complexity to that of the unoptimized baseline. Furthermore, distributed implementations are non-trivial due to the high communication overhead of graph traversals across partitions. However, we note that the query processing phase is inherently parallelizable, as queries are

read-only and independent, allowing for high throughput on multi-core systems.

6 Experiments

We now present the experimental results. We first discusses the setup in Section 6.1. Then we report the efficiency results in Sections 6.2 and 6.3. We further motivate the window-CC/SCC by a case study of anomaly DBLP data detection in Section 6.4. Our codes are available in two GitHub repositories including window-CC[2] and window-SCC[3] solutions, respectively.

6.1 Setup

We use 11 real-world temporal graphs from SNAP [43] and KONECT project [39], along with one synthetic graph generated from LDBC [17]. Table 3 provides the statistics of each graph, where n and m are the numbers of vertices and edges respectively, t_{max} is the maximum timestamp (counting from 0), and *size (bytes)* is the memory cost (in bytes) of the graph. The graph500-23 dataset does not have timestamps, and we randomly generate timestamps for it.

To measure the query efficiency of each dataset, for each dataset, we generate 1000 queries with the size of query window (i.e., $t_e - t_s$) as $0.8t_{max}$ where t_s is selected randomly. Then, we execute the 1000 queries sequentially and compute the average time cost of the queries. Besides, the indexing time and the average query time of each dataset is measured, respectively. All algorithms are implemented in C++, compiled with the g++ compiler at -O3 optimization level, and run on a Linux machine with an Intel Xeon 2.40GHz CPU and 512GB RAM.

6.2 Efficiency on undirected temporal graphs

6.2.1 Query processing

Figure 10 shows the average query time of four query algorithms on all undirected datasets.

Note that the results of the U-baseline-query algorithm are only reported for the first three datasets since it raises OOM (out-of-memory) exceptions for the other three datasets because of the huge space cost. In the figures and tables, we use "N/A" to denote that the index construction could not be finished within 72 hours or raised an OOM exception. Besides, although the algorithm implementing the TSF-query with the priority search tree, denoted as PST-query,

Table 3 Datasets used in our experiments

Datasets	Undirected temporal graphs						Directed temporal graphs					
	contact (CT)	mit (MIT)	youtube (YT)	wikipedia (WK)	stackoverflow (ST)	graph500-23 (GR)	bitcoin (BTC)	dblp-cite (DC)	CollegeMsg (CM)	email-eu (EE)	flickr (FK)	amazon (AM)
n	275	97	3,223,590	1,870,710	6,024,271	4,610,223	5,881	12,590	1,899	986	2,302,926	19,546,448
m	28,244	1,086,404	9,375,374	39,953,145	63,497,050	129,333,677	35,592	49,759	59,835	332,334	33,140,017	52,885,992
t_{max}	15,661	33,451	202	2,197	41,484,768	6,807,035	35,591	29	58,910	207,879	133	272
size (bytes)	225,952	8,691,232	75,002,992	319,625,160	383,226,128	1,034,669,416	284,736	396,632	478,680	2,658,672	265,120,136	423,087,936

[2] https://github.com/ForwardStar/spannedCC

[3] https://github.com/ForwardStar/spannedSCC

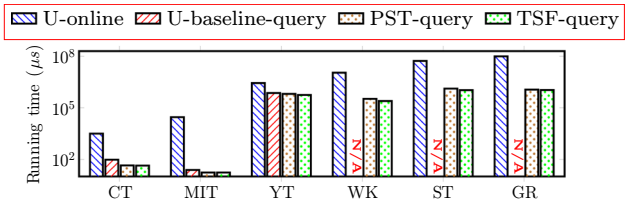


Fig. 10 Average time cost of window-CC queries

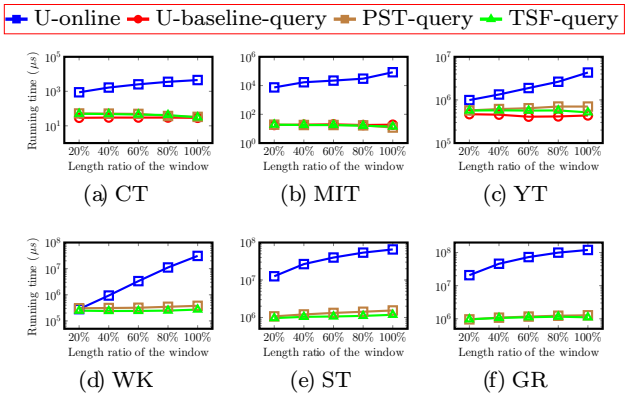


Fig. 11 Effect of the size of the query window on undirected temporal graphs

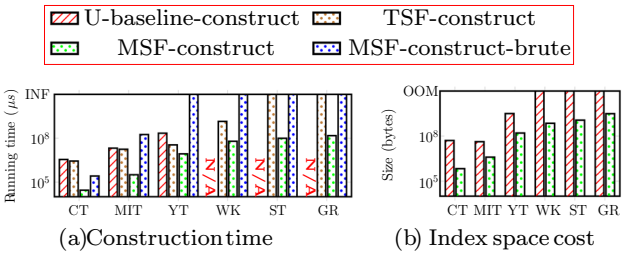


Fig. 12 Index construction time and index space cost

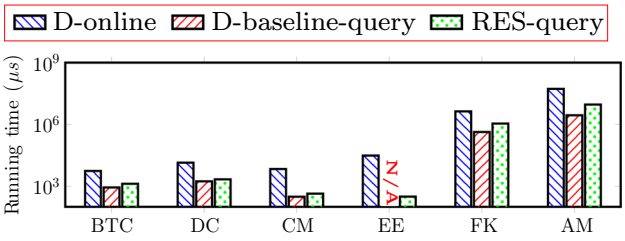


Fig. 13 Average time cost of window-SCC queries

has the smallest theoretical time complexity among all the solutions, the TSF-query implemented by our proposed TSF-index achieves the best efficiency in all datasets.

We also evaluate the effect of the query time window size. For each graph, we fix the start time of the query window as $t = 0$ and consider five query time window sizes, i.e., 20%, 40%, 60%, 80% and 100% of t_{max} respectively. We then record the average running time of 1,000 queries with each window size. Figure 11 reports the efficiency results on undirected datasets. When the time window is small, all algorithms cost a similar amount of time. However, when the time window is larger, the U-online algorithm takes more time, while our index-based solutions' time costs do not change much.

6.2.2 Index construction

We compare the efficiency of index construction algorithms on all graphs in Figure 12(a) . Clearly, the TSF-index can be built faster than the U-baseline index, since U-baseline-construct costs $O((m + n \log n)t_{max})$ time while TSF-construct needs $O(mt_{max})$ time. Besides, our newly proposed MSF-construct algorithm outperforms the original TSF-construct algorithm on all datasets since it only requires $O(m \log n)$ time. Note that for the ST and GR datasets, the original TSF-construct algorithm cannot finish within 72 hours. We further report the space cost of each index on all graphs in Figure 12(b) . Since the construction result of MSF-construct is the same as the original TSF-construct, we only draw one bar to represent the TSF-index space cost in Figure 12(b). We see that the space cost of TSF-index is much less than that of U-baseline-index, because their space costs are bounded by $O(m)$ and $O(n \log n \cdot t_{max})$, respectively.

Besides, we test the scalability of the index construction algorithms. Specifically, for each graph, we build four subgraphs using edges with timestamps in $[0, f \cdot t_{max}]$ with f=25%, 50%, 75%, and 100%, respectively. Table 4 depicts the indexing time and space costs on these subgraphs. We can see that TSF-index scales better than the U-baseline index for its optimized time and space complexities.

We also evaluate the efficiency of index maintenance. Specifically, we first build the U-baseline index and TSF-index with edges in $[0, 0.8t_{max}]$. Then, we update the indices with the remaining edges in $(0.8t_{max}, t_{max}]$. Table 5 shows the average update time of each new edge on each dataset. Clearly, the maintenance of the indices is much faster than rebuilding the indices from scratch.

In addition, the above index-based query algorithms' efficiency is measured without considering the index construction time, so they may not be reasonable for scenarios that need to consider the indexing time and query time in a collective manner. Thus, we also evaluate the number of TSF-index-based queries required to amortize the index construction time cost and report the minimum numbers of such queries in Table 6. For each dataset, when the total number of queries exceeds the number in Table 6, the TSF-query algorithm will be faster than the U-online algorithm. Note that the results of the U-baseline-index are skipped since it fails to build the index on some datasets.

Table 4 Scalability test of indexing time and size

Metrics	Construction time (μs)							
Indices	U-baseline-construct				MSF-construct			
f	25%	50%	75%	100%	25%	50%	75%	100%
CT	349K	1.1M	2.1M	3.6M	9K	17K	24K	30K
MIT	2.9M	7.6M	15M	22M	130K	225K	281K	318K
YT	19M	44M	96M	221M	250K	405K	615K	1.1M
WK	57M	N/A	N/A	N/A	680K	4M	20M	68M
ST	N/A	N/A	N/A	N/A	21M	48M	76M	106M
GR	N/A	N/A	N/A	N/A	40M	86M	126M	169M
Metrics	Index space cost (bytes)							
Indices	U-baseline-construct				MSF-construct			
f	25%	50%	75%	100%	25%	50%	75%	100%
CT	5.3M	12M	26M	51M	204K	370K	517K	660K
MIT	12M	22M	35M	43M	1.3M	2.4M	3.2M	3.7M
YT	244M	626M	1.5B	3.1B	43M	63M	93M	152M
WK	429M	N/A	N/A	N/A	7M	52M	220M	677M
ST	N/A	N/A	N/A	N/A	269M	558M	854M	1.1B
GR	N/A	N/A	N/A	N/A	776M	1.6B	2.3B	3.1B

Table 5 The average update time of each new edge

Algorithms	U-baseline-index					
Datasets	CT	MIT	YT	WK	ST	GR
Time (μs)	234	257	59	N/A	N/A	N/A
Algorithms	TSF-index					
Datasets	CT	MIT	YT	WK	ST	GR
Time (μs)	0.98	0.23	1.57	2.21	2.44	1.32

Table 6 The minimum number of queries to amortize the index construction time cost for undirected graphs.

Datasets	CT	MIT	YT	WK	ST	GR
No.	8	3	3	5	2	2

For each dataset, the number of such queries is tiny. As a result, the TSF-query algorithm performs better than the U-online algorithm for applications even with a small number of queries.

To demonstrate the long-term sustainability of our maintenance algorithms, Table 7 reports the break-even thresholds for edge insertions. These values confirm that our incremental update strategy remains significantly more efficient than index reconstruction, even after a large number of graph modifications.

Table 7 The maximum threshold of edge insertions where incremental updating outperforms index reconstruction for undirected graphs.

Datasets	CT	MIT	YT	WK	ST	GR
Threshold	30K	1M	7M	30M	43M	100M

6.3 Efficiency on directed temporal graphs

6.3.1 Query processing

Figure 13 shows the average query time of three query algorithms on all directed datasets. The results of the D-baseline-query algorithm are not reported for the EE dataset since it fails to build the index within 72 hours. Our RES-query algorithm is always faster than the D-online algorithm. The speedup mainly depends on the ratio of m over n as the D-online algorithm and index-based algorithms cost $O(m)$ and $O(n+\sqrt{t_{max}})$ time, respectively. The D-baseline-query algorithm is slightly faster than the RES-query algorithm, which aligns with their time complexities. Besides, the retrieved edge set of RES-index is twice the number of D-baseline-index, and the RES-query algorithm needs to run an SCC algorithm while the D-baseline-query algorithm does not.

We also evaluate the effect of the query time window size by setting experiments similar to those of undirected temporal graphs in Section 6.2.1. Figure 14 depicts the results on directed datasets. Clearly, for any size of the query windows, our index-based solutions always perform better than the D-online solution.

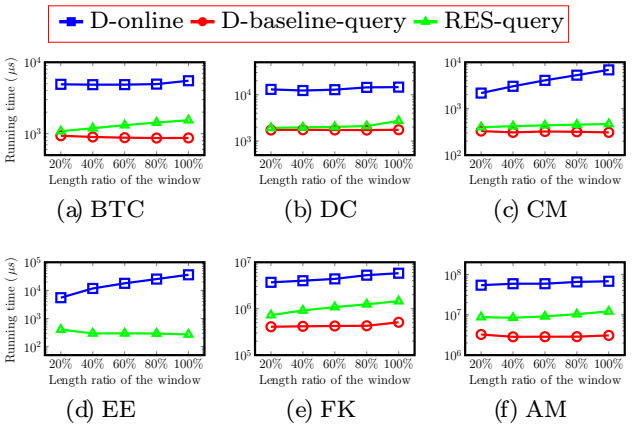


Fig. 14 Effect of the size of the query window on directed temporal graphs

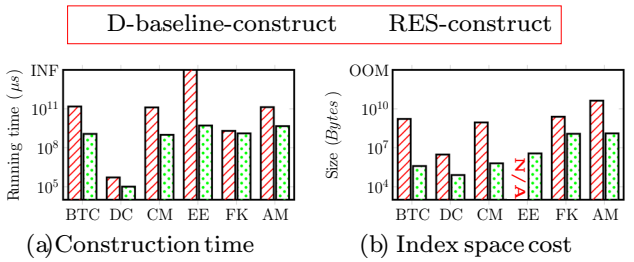


Fig. 15 Index construction time and size

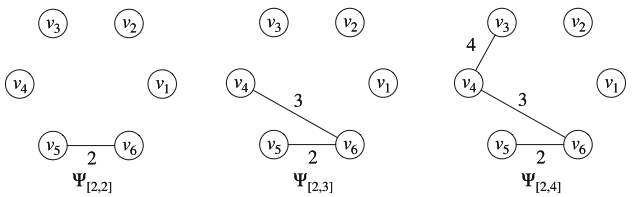


Fig. 16 Index construction time and size

6.3.2 Index construction

We report the efficiency of the index construction algorithms on all datasets in Figure 15(a) . Notice that for EE dataset with a large t_{max} value, the D-baseline-construct algorithm fails to build the index within 72 hours while the RES-construct algorithm succeeds. The reason is that the construction of the D-baseline-index takes $O((m+n)t_{max}^2)$ time, while the time complexity of the RES-index is smaller than it. Besides, we compare the space cost of indices in Figure 15(b) . Clearly, the RES-index costs much less space than the D-baseline-index, since its space cost O(m) is linear to size of the graph, while the D-baseline-index needs $O(nt_{max})$ space.

Besides, we test the scalability of index construction algorithms by setting experiments similar to those for undirected temporal graphs in Section 6.3.2. Table 8 reports the time and space costs for different f values. We can observe that

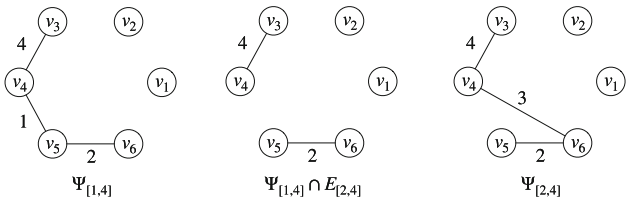


Fig. 17 Construction time

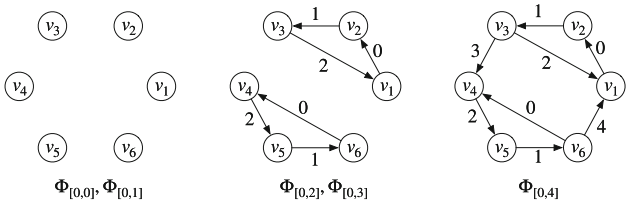


Fig. 18 Index space cost

the RES-index scales better than D-baseline-index because of its optimized time and space complexities.

We also evaluate the maintenance efficiency using similar settings for the efficiency evaluation of index maintenance on the undirected graph. Table 9 shows the average update time of each new edge on each dataset. We see that the maintenance of the indices is much faster than rebuilding the indices from scratch.

In addition, we also evaluate the number of index-based queries that are required to amortize the index construction time cost.

Table 10 presents the minimum number of such queries by using the RES-index on the directed datasets. Note that the results of the D-baseline index are skipped since it fails to build the index on some datasets. We see that if t_{max} is not too large, the RES-query algorithm is more efficient when the number of queries is large.

Table 11 presents the break-even threshold for edge insertions in the directed case. It depicts that our RES-index maintenance strategy supports a substantial volume of updates before a full index reconstruction becomes necessary.

6.4 A case study of anomaly DBLP data detection

Consider the DBLP-citation (DC) graph, where each vertex is a paper and each edge indicates one paper cites the other, and the timestamps are publishing time. Strictly speaking, there should be no SCC in the graph since it implies wrong citations, i.e., a paper published in the year t may cite another paper published in year t' with $t' > t$, but in practice some anomaly cases may exist somehow.

By varying the query time windows of window-SCC queries, we identify two groups of window-SCCs, which reveal the existence of some "wrong" citation data. The

Table 8 Scalability test of indexing time and size.

Metrics	Construction time (μs)							
Indices	D-baseline-construct				RES-construct			
f	25%	50%	75%	100%	25%	50%	75%	100%
BTC	8.6B	34B	72B	152B	41M	229M	711M	1.2B
DC	23K	101K	291K	523K	3K	14K	66K	102K
CM	8.3B	31B	65B	130B	65M	262M	650M	998M
EE	40B	137B	N/A	N/A	562M	1.7B	3.2B	5.2B
FK	144M	466M	1.2B	2.1B	257M	450M	796M	1.2B
AM	2.1B	9.1B	28B	136B	15M	133M	540M	4.7B
Metrics	Index space cost (bytes)							
Indices	D-baseline-construct				RES-construct			
f	25%	50%	75%	100%	25%	50%	75%	100%
BTC	419M	838M	1.3B	1.7B	269K	522K	773K	1M
DC	806K	1.6M	2.3M	3.1M	5.8K	32K	103K	204K
CM	224M	448M	672M	896M	341K	784K	1.2M	1.6M
EE	418M	8.4B	N/A	N/A	2.2M	4.5M	6.8M	9.2M
FK	627M	1.3B	1.9B	2.5B	14M	190M	240M	299M
AM	11B	22B	33B	43B	4M	18M	55M	262M

Table 9 The average update time of each new edge

Algorithms	D-baseline-index					
Datasets	BTC	DC	CM	EE	FK	AM
Time (μs)	5.6M	57K	70M	N/A	37M	2B
Algorithms	RES-index					
Datasets	BTC	DC	CM	EE	FK	AM
Time (μs)	570K	13K	30K	630K	7M	70M

Table 10 The minimum number of queries to amortize the index construction time cost for directed graphs

Datasets	BTC	DC	CM	EE	FK	AM
No.	213K	7	145K	168K	301	88

Table 11 The maximum threshold of edge insertions where incremental updating outperforms index reconstruction for directed graphs.

Datasets	BTC	DC	CM	EE	FK	AM
Threshold	2K	8	302	8.2K	176	60

first group of window-SCCs has time windows of one or two years, while the second group has time windows of three or more years. With a careful investigation, we find that for the first group, the paper citations are actually correct, because some papers may cite other preprint papers which are published in the following one or two years. However, for the second group, we do find some erroneous data. For example, the webpage of DBLP website[4] shows that the SIGMOD paper [28] published in 1989 cites another paper [49] published in 2011. Upon manual verification, we confirmed the cited paper was actually published in 1986, revealing a metadata error on the DBLP website captured by our window-SCCs.

[4] https://dblp.org/rec/conf/sigmod/GraefeW89.html (click "load references" checkbox)

7 Related works

CC queries on static graphs. As a fundamental problem in network science, the computation of CC has been well-studied. Computing CCs in a static graph can be achieved in linear time regarding the input graph size by BFS or DFS [42]. There are some parallel approaches for processing large graphs, such as earlier solutions with the PRAM model [4], and distributed CCs computation algorithms based on MapReduce [54, 57] and Pergel [22, 59]. For the directed graph, computing the SCCs is known to take $O(m + n)$ time using Tarjan algorithm [60] or Kosaraju-Sharir algorithm [64], which are based on DFS. However, these serial algorithms are inefficient for large graphs since the computation of DFS is known P-complete [58]. As a result, many parallel algorithms have been proposed to speed up the computation of SCCs, such as Forward-Backward-based algorithms [23, 32, 62], GPU-based algorithms [46, 69], and

parallel randomized DFS algorithm [47]. Distributed algorithms have also been studied for large directed graphs [36, 72].

Nevertheless, these works mainly focus on static graphs. Their solutions cannot be applied to processing large temporal graphs because of temporal edges.

CC queries on temporal graphs. Recently, a few CC queries have been studied in temporal graphs. For example, [2, 66] proposed the persistent CC model for temporal graphs, which is a set of vertices that are connected in each timestamp of a time interval. [7, 8] introduced the temporal CC model, which is a set of vertices such that each pair of vertices is connected by a path with edges of increasing timestamps. Nevertheless, these works fail to capture the relationships between entities involving the same group or activity with no time-respecting path connecting them. To the best of our knowledge, our work firstly studies window-CC and window-SCC queries in temporal graphs.

CC queries on dynamic graphs. Another kind of related but different graph model is the dynamic graph, where an edge in it may appear in a large time interval, while an edge of a temporal graph only appears at a single timestamp. Some works have studied CC queries on dynamic graphs [6, 31, 56, 65]. A dynamic graph can be transformed into a temporal graph by introducing artificial timestamps, but it may lead to a huge number of new edges and high time cost if using our algorithms.

In the literature, to process temporal and dynamic graphs, many different types of algorithms have been developed, including streaming algorithms [10], semi-streaming algorithms [21], sliding window-based algorithms [14], and incremental graph algorithms [19]. The high-level ideas of building our indices are somewhat similar to the ideas in the above works. That is, when constructing the indices, we anchor each start time and then process the temporal edges incrementally, during which time windows with the same start time have been considered. However, these works do not build indices to compactly organize all the window-CCs of a temporal graph, for supporting CC/SCC queries with arbitrary time windows. It is also worth noting that significant progress has been made on connectivity queries in fully dynamic graphs. For instance, Huang et al. [33] proposed a fully dynamic connectivity algorithm with $O(\log n(\log\log n)^2)$ amortized expected update time, and Xu et al. [41] achieved constant-time connectivity querying. However, there is a fundamental distinction between these works and our paper that these prior works primarily address the pairwise connectivity problem (i.e., determining whether a path exists between two specific vertices u and v), returning a boolean result. In contrast, our work investigates the Connected Component (CC) query, which aims to retrieve the entire maximal set of mutually reachable vertices within a query window.

Other temporal graph queries. Some other queries have been studied on temporal graphs, such as the shortest path problem [34], the reachability problem [68], and the communication motifs [30]. There are also some cohesive subgraph queries studied in temporal graphs, such as k-core queries [45, 52, 70], quasi-clique query [73], and dense subgraph query [48].

8 Conclusion

In this paper, we introduce the concepts of window-CCs and window-SCCs on undirected and directed temporal graphs, respectively. To efficiently answer the window-CC and window-SCC queries, we develop both efficient online algorithms and index-based algorithms, where the best indices cost space linear to the sizes of the temporal graphs. We also develop index maintenance algorithms. The extensive experimental evaluation on 11 real-world datasets and one synthetic dataset demonstrates the high efficiency and effectiveness of the proposed solutions. Finally, we suggest several future explorations: (1) developing indices that are friendly to parallel and distributed settings; (2) proposing efficient algorithms to support edge deletions on the RES-index.

References

1. Afshani, P., Sheng, C., Tao, Y., Wilkinson, B.T.: Concurrent range reporting in two-dimensional space. In Proceedings of the twenty-fifth annual ACM-SIAM Symposium on Discrete algorithms 983–994 . SIAM, (2014)
2. Akrida, E.C., Spirakis, P.G.: On verifying and maintaining connectivity of interval temporal networks. Parallel Processing Letters **29**(02), 142–154 (2019)
3. Allard, G., Jacquet, P., Mans, B.: Routing in extremely mobile networks. In K. Al Agha, I. Guérin Lassous, and G. Pujolle, editors, Challenges in Ad Hoc Networking 129–138 (2006)
4. Awerbuch, B., Shiloach, Y.: New connectivity and msf algorithms for shuffle-exchange network and pram. IEEE Trans. Comput. **36**(10), 1258–1263 (1987)
5. Beamer, S., Buluç, A., Asanovic, K., Patterson, D.: Distributed memory breadth-first search revisited: Enabling bottom-up search. In 2013 IEEE International Symposium on Parallel and Distributed Processing. Workshops and Phd Forum 1618–1627 (2013)
6. Bernstein, A., Probst, M., Wulff-Nilsen, C.: Decremental strongly-connected components and single-source reachability in near-linear time. In ACM SIGACT 365–376 (2019)
7. Bhadra, S., Ferreira, A.: Complexity of connected components in evolving graphs and the computation of multicast trees in dynamic networks. In Samuel Pierre, Michel Barbeau, and Evangelos Kranakis, editors, Ad-Hoc, Mobile, and Wireless Networks 259–270 (2003)
8. Bhadra, S., Ferreira, A.: Computing multicast trees in dynamic networks and the complexity of connected components in evolving

graphs. Journal of Internet Services and Applications **3**(3), 269–275 (2012)
9. Cai, T., Cheng, D., Liang, C., Liu, Z., Gu, L., Xie, H., Zhang, Z., Zeng, X., Gu, J.: Linklouvain: Link-aware a/b testing and its application on online marketing campaign. In Christian S. Jensen, Ee-Peng Lim, De-Nian Yang, Wang-Chien Lee, Vincent S. Tseng, Vana Kalogeraki, Jen-Wei Huang, and Chih-Ya Shen, editors, *Database Systems for Advanced Applications*, pages 499–510, Cham, Springer International Publishing. ISBN 978-3-030-73200-4 (2021)
10. cGregor, A.M.: Graph stream algorithms: A survey. *SIGMOD Rec.*, 43 (1): 9–20, (2014) ISSN 0163-580 https://doi.org/10.1145/2627692.2627694
11. Chen, X., Wang, K., Lin, X., Wenjie Zhang, L., Qin, Ying, Z.: Efficiently answering reachability and path queries on temporal bipartite graphs. Proc. VLDB Endow. **14**(10), 1845–1858 (2021a). https://doi.org/10.14778/3467861.3467873
12. Chen, X., Wang, K., Lin, X., Wenjie Zhang, L., Qin, Zhang, Y.: Efficiently answering reachability and path queries on temporal bipartite graphs. Proc. VLDB Endow. **14**(10), 1845–1858 (2021). https://doi.org/10.14778/3467861.3467873
13. Chowdhury, A., Rahman, J.S., Hawlader, M.S.: Well-connectedness - a novel measure for improving protein complex detection from ppi network. In *2016 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*, 1–6 (2016)
14. Crouch, M.S., McGregor, A., Stubbs, D.: Dynamic graphs in the sliding-window model. In Hans L. Bodlaender and Giuseppe F. Italiano, editors, *Algorithms – ESA 2013*, pages 337–348, Berlin, Heidelberg, Springer Berlin Heidelberg. ISBN 978-3-642-40450-4 (2013)
15. Cybenko, G., Allen, T.G., Polito, J.E.: Practical parallel union-find algorithms for transitive closure and clustering. Int. J. Parallel Program. **17**(5), 403–423 (1989)
16. Daniel Dominic Sleator and Robert Endre Tarjan: Self-adjusting binary search trees. Journal of the ACM (JACM) **32**(3), 652–686 (1985)
17. Erling, O., Averbuch, A., Larriba-Pey, J., Chafi, H., Gubichev, A., Prat, A., Pham, M.-D., Boncz, P.: The ldbc social network benchmark: Interactive workload. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 619–630 (2015)
18. Fabijan, A., Dmitriev, P., Olsson, H.H., Bosch, J.: Online controlled experimentation at scale: An empirical survey on the current state of a/b testing. In *2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 68–72, (2018) https://doi.org/10.1109/SEAA.2018.00021
19. Fan, W., Tian, C.: Incremental graph computations: Doable and undoable. ACM Trans. Database Syst. **47**(2) (2022). https://doi.org/10.1145/3500930
20. Fang, Y., Xin Huang, L., Qin, Y.Z., Zhang, W., Cheng, R., Lin, X.: A survey of community search over big graphs. VLDB J. **29**(1), 353–392 (2020)
21. Feigenbaum, J., Kannan, S., McGregor, A., Suri, S., Zhang, J.: On graph problems in a semi-streaming model. In Josep Díaz, Juhani Karhumäki, Arto Lepistö, and Donald Sannella, editors, *Automata, Languages and Programming*, pages 531–543, Berlin, Heidelberg, Springer Berlin Heidelberg. ISBN 978-3-540-27836-8 (2004)
22. Feng, X., Chang, L., Lin, X., Qin, L., Zhang, W.: Computing connected components with linear communication cost in pregel-like systems. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, pages 85–96. IEEE, (2016)
23. Fleischer, L.K.: Bruce Hendrickson, and Ali Pınar. On identifying strongly connected components in parallel. In *International Parallel and Distributed Processing Symposium*, pages 505–511. Springer, (2000)
24. Fox, A.D., Hescott, B.J., Blumer, A.C., Slonim, D.K.: Connectedness of PPI network neighborhoods identifies regulatory hub proteins. Bioinformatics **27**(8), 1135–1142 (2011)
25. Frederickson, G.N.: Data structures for on-line updating of minimum spanning trees. In Proceedings of the fifteenth annual ACM symposium on Theory of computing 252–257 (1983)
26. Gabow, H.N.: Path-based depth-first search for strong and biconnected components. Inf. Process. Lett. **74**(3), 107–114 (2000)
27. Gabow, H.N., Myers, E.W.: Finding all spanning trees of directed and undirected graphs. SIAM J. Comput. **7**(3), 280–287 (1978)
28. Graefe, G., Ward, K.: Dynamic query evaluation plans. In *Proceedings of the 1989 ACM SIGMOD international conference on Management of data*, 358–366 (1989)
29. Gui, H., Xu, Y., Bhasin, A., Han, J.: Network a/b testing: From sampling to estimation. In *Proceedings of the 24th International Conference on World Wide Web*, WWW '15, page 399–409, Republic and Canton of Geneva, CHE, 2015. International World Wide Web Conferences Steering Committee. ISBN 978145033469https://doi.org/10.1145/2736277.2741081
30. Gurukar, S., Ranu, S., Ravindran, B.: Commit: A scalable approach to mining communication motifs from dynamic networks. In SIGMOD 475–489 (2015)
31. Haryan, C.A., Ramakrishna, G., Kothapalli, K., Banerjee, D.S.: Shared-memory parallel algorithms for fully dynamic maintenance of 2-connected components. In *IPDPS*, pages 1195–1205. IEEE, (2022)
32. Hong, S., Rodia, N.C., Olukotun, K.: On fast parallel detection of strongly connected components (scc) in small-world graphs. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–11, (2013)
33. Huang, S.-E., Huang, D., Kopelowitz, T., Pettie, S.: Fully dynamic connectivity in o (log n (log log n) 2) amortized expected time. In Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms 510–520 (2017)
34. Huanhuan, W., Cheng, J., Huang, S., Ke, Y., Yi, L., Yanyan, X.: Path problems in temporal graphs. Proceedings of the VLDB Endowment **7**(9), 721–732 (2014)
35. Jacquet, P., Mans, B.: Routing in intermittently connected networks: Age rumors in connected components. In *Fifth Annual IEEE International Conference on Pervasive Computing and Communications Workshops (PerComW'07)*, pages 53–58, (2007)
36. Ji, Y., Liu, H., Howie, H., Huang.: ispan: Parallel identification of strongly connected components with spanning trees. In SC18: International Conference for High Performance Computing. Networking, Storage and Analysis 731–742 . IEEE, (2018)
37. Khurana, U., Deshpande, A.: Efficient snapshot retrieval over historical graph data. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, pages 997–1008, (2013)
38. Kontogiannis, S., Zaroliagis, C.: Distance oracles for time-dependent networks. Algorithmica **74**(4), 1404–1434 (2016). https://doi.org/10.1007/s00453-015-0003-0
39. Kunegis, J.: KONECT - The Koblenz Network Collection. In Proc. Int. Conf. on World Wide Web Companion 1343–1350 (2013)
40. Kural, M.: Tree traversal and word order. Linguistic Inquiry **36**(3), 367–387 (2005)
41. Lantian, X., Dong Wen, L., Qin, R.L., Zhang, Y., Lin, X.: Constant-time connectivity querying in dynamic graphs. Proceedings of the ACM on Management of Data **2**(6), 1–23 (2024)
42. Leiserson, C.E., Rivest, R.L., Cormen, T.H., Stein, C.: *Introduction to algorithms*, volume 3. MIT press, (1994)
43. Leskovec, J., Sosič, R.: Snap: A general-purpose network analysis and graph-mining library. ACM Transactions on Intelligent Systems and Technology (TIST) **8**(1), 1 (2016)
44. Levorato, V., Petermann, C.: Detection of communities in directed networks based on strongly p-connected components. 10 (2011)

45. Li, R.-H., Su, J., Qin, L., Yu, J.X., Dai, Q.: Persistent community search in temporal networks. In *ICDE*, pages 797–808. IEEE,(2018)
46. Li, G., Zhu, Z., Cong, Z., Yang, F.: Efficient decomposition of strongly connected components on gpus. J. Syst. Architect. **60**(1), 1–10 (2014)
47. Lowe, G.: Concurrent depth-first search algorithms based on tarjan's algorithm. Int. J. Softw. Tools Technol. Transfer **18**(2), 129–147 (2016)
48. Ma, S., Hu, R., Wang, L., Lin, X., Huai, J.: Fast computation of dense temporal subgraphs. In *ICDE*, pages 361–372. IEEE, (2017)
49. Madnick, S.E., Infoplex, M.H.: research in a high-performance database computer. (1986)
50. Manne, F., Patwary, Md. M.A.: A scalable parallel union-find algorithm for distributed memory computers. pages 186–195, 09 (2009). ISBN 978-3-642-14389-2
51. McCreight, E.M.: Priority search trees. SIAM J. Comput. **14**(2), 257–276 (1985)
52. Michael, Yu., Dong Wen, L., Qin, Y.Z., Zhang, W., Lin, X.: On querying historical k-cores. Proc. VLDB Endow. **14**(11), 2033–2045 (2021)
53. Michail, O.: An introduction to temporal graphs: An algorithmic perspective. *Internet Mathematics*, 12, 03 201https://doi.org/10.1007/978-3-319-24024-4_18
54. Qin, L., Yu, J.X., Chang, L., Cheng, H., Zhang, C., Lin, X.: Scalable big graph processing in mapreduce. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pages 827–838, (2014)
55. Qiu, X., Cen, W., Qian, Z., Peng, Y., Zhang, Y., Lin, X., Zhou, J.: Real-time constrained cycle detection in large dynamic graphs. Proceedings of the VLDB Endowment **11**(12), 1876–1888 (2018)
56. Rannou, L., Magnien, C., Latapy, M.: Strongly connected components in stream graphs: Computation and experimentations. In *International Conference on Complex Networks and Their Applications*, pages 568–580. Springer, (2020)
57. Rastogi, V., Machanavajjhala, A., Chitnis, L., Sarma, A.D.: Finding connected components in map-reduce in logarithmic rounds. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, pages 50–61. IEEE, (2013)
58. Reif, J.H.: Depth-first search is inherently sequential. Inf. Process. Lett. **20**(5), 229–234 (1985)
59. Salihoglu, S., Widom, J.: Optimizing graph algorithms on pregel-like systems. Proceedings of the VLDB Endowment **7**(7) (2014)
60. Sharir, M.: A strong-connectivity algorithm and its applications in data flow analysis. Computers & Mathematics with Applications **7**(1), 67–72 (1981)
61. Sleator, D.D., Tarjan, R.E.: A data structure for dynamic trees. In Proceedings of the thirteenth annual ACM symposium on Theory of computing 114–122 (1981)
62. Slota, G.M., Rajamanickam, S., Madduri, K.: Bfs and coloring-based parallel algorithms for strongly connected components and related problems. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium*, pages 550–559. IEEE, (2014)
63. Song, J., Wen, D., Lantian, X., Qin, L., Zhang, W., Lin, X.: On querying historical connectivity in temporal graphs. Proceedings of the ACM on Management of Data **2**(3), 1–25 (2024)
64. Tarjan, R.: Depth-first search and linear graph algorithms. SIAM J. Comput. **1**(2), 146–160 (1972)
65. Tench, D., West, E., Zhang, V., Bender, M.A., Chowdhury, A., Dellas, J.A., Farach-Colton, M., Seip, T., Zhang, K.: Graphzeppelin: Storage-friendly sketching for connected components on dynamic graph streams. In *SIGMOD*, pages 325–339. ACM, (2022)
66. Vernet, M., Pigné, Y., Sanlaville, E.: A study of connectivity on dynamic graphs: computing persistent connected components. *4OR*, 04 (2022)
67. Wen, D., Huang, Y., Zhang, Y., Qin, L., Zhang, W., Lin, X.: Efficiently answering span-reachability queries in large temporal graphs. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, 1153–1164, (2020)
68. Wen, D., Yang, B., Ying Zhang, L., Qin, D.C., Zhang, W.: Span-reachability querying in large temporal graphs. VLDB J. **31**(4), 629–647 (2022)
69. Wijs, A., Katoen, J.-P., Bošnački, D.: Efficient gpu algorithms for parallel decomposition of graphs into strongly connected and maximal end components. Formal Methods in System Design **48**(3), 274–300 (2016)
70. Wu, H., Cheng, J., Lu, Y., Ke, Y., Huang, Y., Yan, D., Wu, H.: Core decomposition in large temporal graphs. In *2015 IEEE International Conference on Big Data (Big Data)*, pages 649–658. IEEE, (2015)
71. Xie, H., Fang, Y., Xia, Y., Luo, W., Ma, C.: On querying connected components in large temporal graphs. Proceedings of the ACM on Management of Data **1**(2), 1–27 (2023)
72. Yan, D., Cheng, J., Xing, K., Yi, L., Ng, W., Yingyi, B.: Pregel algorithms for graph connectivity problems with performance guarantees. Proceedings of the VLDB Endowment **7**(14), 1821–1832 (2014)
73. Yang, Y., Yan, D., Huanhuan, W., Cheng, J., Zhou, S., Lui, J.C.S.: Diversified temporal subgraph pattern mining. In SIGKDD 1965–1974 (2016)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.